

A project report on

SYMPHONY OF ALGORITHMS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology In Computer Science and Engineering with specialization in Artificial Intelligence and Machine Learning

by

HAMSHIKA R (21BAI1003)

HARSHITA RAJINIKANTH (21BAI1226)

SNEHALATHA PRAKASH (21BAI1519)



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2024



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled “**SYMPHONY OF ALGORITHMS**” submitted by **HAMSHIKA R (21BAI1003), HARSHITA RAJINIKANTH (21BAI1226), SNEHALATHA PRAKASH (21BAI1519)**, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of **DR. HARINI S.**

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: 06/11/2024

Hamshika R



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled “**Symphony of Algorithms**” is prepared and submitted by **Hamshika R (21BAI1003)** to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science and Engineering with Specialization in Artificial Intelligence and Machine Learning** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr./Prof. Harini S

Date: 13th November 2024

Signature of the Examiner

Name:

Date:

Signature of the Examiner

Name:

Date:

Approved by the Head of Department,
**Computer Science and Engineering with specialization in
Artificial Intelligence and Machine Learning**

Name: **DR. SWEETLIN HEMALATHA**

Date:

(Seal of SCHOOL YOU BELONG To)

ABSTRACT

Artificial intelligence has made tremendous progress in many applications of music composition. The performance comparison of models relating to coherent and structured outputs in musical compositions is still underdeveloped.

Thus, filling this gap, the report of the current project comparatively analyzes the performance of six major deep learning models applied for the generation of music: GANs, GRUs, VAEs, OpenAI's Jukebox, Transformers, and Diffusion Models.

Though all these models produce music, differences arise while applying various architectural approaches. For example, GANs fine-tune outputs through adversarial training, and GRUs capture the ability for temporal dependency with regard to musical flow. VAEs have probabilistic variability and enable creative exploration while Jukebox, instead, utilized the transformer-based architecture for generating highly faithful audio with complex musical patterns. Transformer models and self-attention mechanisms have essentially revolutionized music generation, allowing for the capture of long-range dependencies and intricate structures within compositions. Diffusion models constitute a class of quite novel innovation in generative AI that iteratively refines noise into structured, musically coherent outputs- the smooth transitions and fine-grained detail.

This work compares the based on multiple aspects of quality of music, including harmonic consistency, novelty, coherence, and possible creation of both conventional and innovative forms of musical sounds. GANs have been performing remarkably well with adversarial refinement in more realistic audio outputs, while GRUs have been very good with their ability to model sequential structures in guaranteeing coherent transitions between beats, rhythms, and melodies. VAEs promise creative variability and sometimes at the expense of structural coherence. OpenAI's Jukebox is impressive with high-fidelity audio generation with intricate complex musical patterns close to real-world music. Using their ability to capture long-range dependencies and relationships, transformers enable more fine-tuned compositions with deeper harmonic and melodic levels. Diffusion models are unique in achieving a high level of detail and smooth transitions, and for this reason, high granularity in composing will be much more important in music applications.

That is to say, the main purpose of this paper will be to discuss methodically the strength and weakness of each model with regard not only to technical metrics like harmonic consistency and rhythmic stability but also with regard to subjective human judgment, therefore providing account for that artistic and emotional quality which distinguishes good music

There are three main objectives in this research: to assess, respectively for each model, how well they can create music that, in their own ways, is traditional but innovative; to discover which model best balances creative control with coherence of the produced music as practical music; and lastly, through these models, to discuss some of their more avant-garde applications in non-traditional genres.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Dr. Harini S, Head Of Department of B.Tech. CSE with Specialization in Artificial Intelligence and Robotics, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for her constant guidance, continual encouragement, understanding; more than all, she taught me patience in my endeavor. My association with her is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of Deep Learning.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Ganesan R, Dean, Dr. Parvathi R, Associate Dean Academics, Dr. Geetha S, Associate Dean Research, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to Dr. Sweetlin Hemalatha, Head of the Department, B.Tech. Computer Science and Engineering and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: 13th November 2024

Harshita Rajinikanth

CONTENTS

CONTENTS	III
LIST OF FIGURES	V
LIST OF ACRONYMS	VI
CHAPTER 1	
INTRODUCTION	
1.1 INTRODUCTION	1
1.2 OVERVIEW	3
1.3 CHALLENGES PRESENT	5
1.4 PROJECT STATEMENT	8
1.5 OBJECTIVES	9
1.6 SCOPE OF THE PROJECT	10
CHAPTER 2	
MODEL ARCHITECTURES	
2.1 LITERATURE REVIEW	11
2.2 MODULES USED	16
2.3 ROLE OF MODULES IN MUSIC GENERATION MODELS	18

2.4 MODELS	
2.4.1 GENERATIVE ADVERSERIAL NETWORKS	19
2.4.2 GATED RECURRENT UNITS	23
2.4.3 VARIATIONAL AUTOENCODERS	26
2.4.4 OPEN AI JUKEBOX	29
2.4.5 TRANSFORMERS	32
2.4.6 DIFFUSION MODELS	36

CHAPTER 3

RESULTS AND METRICS

3.1 GENERATIVE ADVERSERIAL NETWORKS	40
3.2 GATED RECURRENT UNITS	45
3.3 VARIATIONAL AUTOENCODERS	47
3.4 TRANSFORMERS	50
3.5 OPEN AI JUKEBOX	54
3.6 DIFFUSION MODELS	55

CHAPTER 4

CONCLUSION

CONCLUSION AND FUTURE WORK	58
APPENDICES	59
REFERENCES	61

LIST OF FIGURES

1.1 VARIATIONAL AUTOENCODER ARCHITECHTURE

1.2 GENERATIVE ADVERSERIAL NETWORKS ARCHITECHTURE

1.3 GATED RECURRENT UNIT

1.4 OPEN AI JUKEBOX

1.5 TRANSFORMERS

1.6 DIFFUSION MODELS

LIST OF ACRONYMS

AI – ARTIFICIAL INTELLIGENCE

GANs – GENERATIVE

ADVERSARIAL NETWORKS

GRU – GATED RECURRENT UNIT

VAEs – VARIATIONAL AUTOENCODERS

JUKEBOX – OPENAI’s JUKEBOX (a model for music generation by openai)

Chapter 1

Introduction

1.1 INTRODUCTION

Composition of music has always been regarded as a means to an end, something that is intensely creative and human, emotional as a storyteller and able to capture the very nuances of culture and experience.

Much more recent significance is the area of artificial intelligence, or AI, and its role within the creative process, particularly related to the generation of music.

This project, "Symphony of Algorithms: Music Composition Using Deep Learning Models," discusses the several state-of-the-art deep learning architectures, including GANs, GRUs, VAEs, transformers, diffusion models, and OpenAI's Jukebox, to obtain AI-driven music composition. Each of these models uses a bit different methodology to create music and therefore focus is set on each different advantage and limitation.

GANs generate music through an adversarial process: the model progressively refines its output through a "generator" and "discriminator" mechanism. This provides the ability of the model to progressively enhance the quality of the music compositions, although the models can indeed struggle with long-term coherence and complexity in musical structure.

In fact, Gated Recurrent Units-the type of recurrent neural network-are designed to capture sequential dependencies; that is, they are really suitable for maintaining temporal flow and structure in musical sequences. Sure, GRUs can do a great job generating music with clear rhythmic and melodic progressions, but their output may sometimes seem not so diverse as from a more complex model.

Variational Autoencoders (VAEs), in this context, apply a probabilistic framework that allows the generation of diverse and creative variations in music. Relatively, its basic strength can be seen in the fact that it explores new compositions and the scope of possible musical possibilities it offers; VAEs sometimes result in less cohesive and less rich music than an audience might expect from more standard compositions.

Transformers is a family of models, initially popularized in natural language processing, now adapted to music generation. Transformers are very good at learning long-range dependencies in data; hence, they make particularly good generators of complex musical structures- intricate counterpoint, harmonies, and long compositions. Their ability to process large sequential data makes it easier to generate in parallel, which usually yields highly coherent and nuanced musical outputs. However, the challenge with transformers is that there are so many creative possibilities, and still, there is a possibility of overfitting or losing control of the content.

Diffusion Models are newly introduced in generative AI for their capabilities to generate high-quality diverse outputs through iterative steps that gradually transform noise into data. The power of diffusion models lies in the generation of novel musical compositions through learning to "denoise" or refine an initial random signal into structured,

harmonic sound. These models are promising because they can generate extremely diverse outputs at high resolution; however, these models remain computationally intensive and relatively less tested compared with more established approaches in the music domain.

Developed by OpenAI, the transformer-based Jukebox is able to create rich high-resolution audio with complex musical patterns that press the current limits of AI in music generation. While symbolic representations such as MIDI operate with abstract models and are incapable of generating enormously complex dynamics, timbres, and textures in the generated music, Jukebox directly operates on raw audio. While capable of producing highly detailed compositions in a plethora of very diverse genres, Jukebox's outputs could sometimes be rather unpredictable and require large computational power to generate high-quality results.

It encompasses a broad category of generative models, including GANs, GRUs, VAEs, transformers, diffusion models, and Jukebox; the research described here offers a broad look at the current state of the art of AI-driven music composition and its potential for the future. We hope our comparison yields useful results in the expanding crossroads of AI and music as we bridge the human creativity gap and machine-generated innovation.

1.2 OVERVIEW

The blending of artificial intelligence (AI) and music composition presents one of the most interesting and progressive fields of recent times. AI models, through their capacity to learn from vast datasets, now generate original music expressing nuances of broad genres, style, and complexity. The biggest challenge in this evolution of models is selecting the appropriate deep learning architecture to produce coherent, high-quality, and creatively diverse music compositions. To address this challenge, the project entitled *Symphony of Algorithms: Music Composition Using Deep Learning Models* explores the effectiveness of several leading AI models in music generation, including GANs, GRUs, VAEs, transformers, diffusion models, and OpenAI's Jukebox. The project seeks to assess these models using a suite of musical quality metrics in order to understand where they are strong, weak, and potential applications in AI-driven music composition.

Generative Adversarial Networks (GANs) GANs constitutes a category of machine learning models with two networks: the generator and the discriminator. The generator creates new information (in this instance, music); the discriminator assesses how realistic it sounds compared to actual examples. Through an iterative process of training, the generator learns to produce more realistic outputs, iteratively refining its compositions. GANs effectively generate highly realistic pieces and original works by engaging in an adversarial process-the generator improves as a result of feedback from the discriminator. On the other hand, although GANs are better at generating convincing music, they can suffer from musical coherence loss in the long run, especially when dealing with more complex compositions.

GRUs are a kind of RNN that is especially designed to capture sequential data and long-range dependencies. For musical composition, this would mean that GRUs are good at keeping the temporal flow of musical sequences intact. They are quite good at producing music that follows a certain rhythmic or harmonic pattern, such as melodic structures, harmonic structures, or chord patterns. GRUs have fewer parameters than the traditional RNNs; therefore, they can be computationally much more efficient with a capability to model long-term dependencies in music. Although GRUs are very strong with regard to keeping continuity in melodic and harmonic structures, they may not produce as diverse or complicated variations as some other models, such as VAEs or transformers.

Variational Autoencoders VAE is a generative model, focusing on learning a probabilistic latent space. Unlike the process of generating a new data point by computation, variational autoencoder can be sampled for creating new data. The application of this technique in music composition enlists creative variations by mapping music into latent space and then back to new compositions. Since it gives a probabilistic approach, VAE tends to encourage diversity in the generated music: this makes it ideal for exploration toward novel musical patterns, variations, and combinations. Such can create unique pieces that diverge from the input data, given that VAEs learn the mapping of music into a lower-dimensional latent space. Sometimes, however, VAEs fail to carry the same structure and coherence like those for transformers or Jukebox.

Transformers, Transformed natural language processing and changed music generation forever. These models will be excellent at getting long-range dependencies and at processing sequential data in parallel, thereby generating very complex and coherent musical compositions. Applications in music: transformers can well handle rich musical structures by parameterizations of complex harmonies, counterpoint, and rhythmic patterns. They are particularly well suited to modeling long-term musical relations, which characterize extended compositions or complex patterns in time. Transformers are quicker and more flexible, as they can process enormous datasets in parallel, although overfitting or losing control to the complexities of data easily happens while focused on creative diversity, not at the expense of musical coherence.

Diffusion Models Recently, diffusion models have emerged as a novel class of generative models that have demonstrated immense promise in a range of domains, including image and audio synthesis. These models denoise noise into structured data through a series of iterative steps, effectively “denoising” the data over time. The technique for music generation with diffusion models can start with a random noise signal and iteratively refine it into a coherent musical piece. This allows for creating high-quality, diverse outputs by learning to structure complex patterns from random noise. Diffusion models are particularly well-suited for generating new music compositions that enter unexplored creative territory since it can yield extremely high-resolution and innovative results. However, they are computationally expensive to train, and similar to the VAE case, they require a lot of training data in order to produce the best results. Maintaining long-term coherence in novel composition generation is still an area of research.

Jukebox is a transformer-based model developed by OpenAI that has the capability of developing high-resolution music and actual compositions with or without vocals. Unlike other models, Jukebox produces not only music but also raw audio waveform to improve high-fidelity audio production. The architecture of Jukebox exploits the power of transformers because of their ability to capture long-range dependencies in data, which puts it at a good position to handle complex musical patterns. Here, the power of Jukebox to create high-quality audio at a granular level is a breakthrough regarding AI music generation; therefore, it serves as a platform for developing highly detailed and expressive musical work. However, the outputs of Jukebox are sometimes quite unpredictable, along with its potential for consuming significant computational resources that makes the process a bit hard for composers and researchers to fine-tune.

1.3 CHALLENGES PRESENT

1. Model Complexity and Training Needs

- Highly Computationally Intensive:

Training deep learning models, especially large models such as OpenAI's Jukebox, transformers like GPT-style models, and diffusion models requires extensive computational resources. Such models do require high-performance hardware such as GPUs or TPUs to be trained efficiently. For instance, transformers use giant memory capacities and parallel processing power since it supports long-range dependency across a music sequence. Diffusion models, which involve progressive refinement of noise into coherent music, also require significant computations and sometimes substantial hardware resources to train and deploy models at inference time.

- Computation Time:

Models of deep learning, like GANs, VAEs, transformers, and diffusion models, require training on large datasets for thousands, tens of thousands, or more musical compositions. Training times can be exceptionally long, notably for transformers whose use of attention mechanisms empowers modeling of complex dependencies over long ranges in sequential data. Millions of parameters in these models and the need to fine-tune their hyperparameters and optimize for better performance lead to lengthy periods of training. Even diffusion models are not exempt from this; usually, they need many forward and reverse steps to yield good-quality outputs, which makes them also time-consuming.

2. Data Collection and Preprocessing

- Diversified and High-Quality Datasets:

As is the case with all deep learning models, transformers and diffusion models benefit from diverse and high-quality datasets upon which to learn. Curation of datasets for transformers in particular is challenging because such considerable sequential data is required to capture rich relationships between musical notes, tempos, and structures. Preprocessing raw data into formats suitable for these models can be complex-with tokenizing music into meaningful units for transformers or defining a suitable noise schedule for diffusion models.

- Preprocessing Issues

For transformer-based models that take raw audio as input (such as Jukebox), extra preprocessing steps may be required to convert the waveforms into sequences that the model accepts.

For example, converting waveforms into spectrograms or mel-spectrograms. Analogous to that, diffusion models also require setting up a noise process and its inverse process that would do generation. The variety of such data preprocessing can be very complex in dealing with multiple forms of music or complex musical structures.

3. Evaluation Metrics and Subjectivity

- Quantitative metric drawbacks:

Therefore, metrics like harmonicity, novelty, and rhythmic consistency based on quantitative measures may not fully represent the complexity inherent in creative compositions. It might be very deceptive to use numerically computable

4. metrics, as shown in the case of transformers and diffusion models for AI-generated music, which can very easily produce complex, very intricate, and detailed outputs. Although transformers can capture the long-range dependencies in music really well, they may generate more sequences that cannot be measured by any conventional metrics, which have emotional depth.
 - Appraisal with Human Subjectivity:
The human evaluation of AI-composed music provides a great insight into such matters but is too subjective in nature. When one works with transformer models and diffusion models, for instance, whose outcomes vary highly and unpredictably, the notion of subjective judgment becomes exceedingly challenging. For, listeners hear or perceive the music differently because some are either familiar with the style or possess personal preference for harmony, melody, or rhythm, hence making the evaluation even more nuanced.
5. Coherence and Creativity Balance
 - The right balance:
Attention mechanisms in transformers generally do well on maintaining coherence even in very long sequences of music, but at the cost of sacrificing creativity for the sake of structure. Iterative refinement in diffusion models can result in outputs that are extremely creative and novel but might also often end up being unstructured in the sense that it lacked structural integrity. It is hence challenging to balance creativity with coherence to ensure that the music sounds both new and musically valid.
 - Overfitting or Poor Generalization:
Even diffusion models can overfit, especially when the data they are being trained on is limited or unrepresentative in some way. Transformers, learning to model dependencies between tokens, often generate music that sounds rather like the training data and therefore does not have a great deal of novelty. Diffusion models will tend to produce unrealistic results if the model does not learn suitable denoising steps or generalizes poorly from training data. It has been challenging to figure out models not to overfit while still making fresh compositions.
6. Musical Diversity and Styles
 - Genre Adaptation Models:
Transformer-based models of GPT-3 that produce coherent sequences across styles and genres might fail in certain music attributes such as those that are exclusive to a genre, like jazz improvisation or electronic production techniques because transformers learn a generalized distribution over musical patterns hence failing to capture unique nuances of a given genre. Diffusion models are somewhat more problematic where specific genre details are concerned because they produce music by gradually refining noise into patterns, thereby limiting their cross-genre adaptability to genres like classical or jazz that require more intricate attention to harmony and form.
 - Capturing Cultural Contexts:
Although both transformers and diffusion models can produce a variety of genres of music, machines basically fail to discover the depth of cultural and

emotional contexts that different musical styles hold within themselves. For example, transformer models trained on worldwide datasets might learn features of a certain style but would miss the cultural nuances or history imbedded in African drumming or Indian classical music. Diffusion models also can produce such compositions that may sound structurally valid but may fail to evoke the right kind of cultural significance or emotional depth of a particular genre of music.

7. Interpretability and Transparency of AI Models

• Black-Box Nature of Deep Learning Models:

Transformers and diffusion models are, like most other deep learning models, black boxes in the sense that one finds extreme difficulty in comprehending how these models generate any particular musical output for a given input. It is possible to interpret, sometimes, what parts of the sequence the transformer is focusing on by the attention mechanism, but there is still no clearness about why some decisions are made. Similar is the obscurity of diffusion models; namely, it ceases not quite to be comprehensible how the intermediate steps connect the noisy input and the final musical output, and hence the diagnosis of why a model fails or succeeds in generating coherent music is difficult.

• Lack of Predictability About Outputs

While transformers are mostly deterministic, this allows them to still generate unpredictable outputs under the right input conditions and model randomness. Diffusion models, being inherently stochastic, are much more unpredictable in comparison: noise generated at early stages of generation produces highly varied outputs. Predictability is a two-edged sword: while creativity is encouraged, it makes it more challenging to guarantee that a given model will produce musically coherent and expected results on an everyday basis.

8. Model Integration and Scalability

• Model Integration and Workflow

It is quite challenging to integrate models, such as transformers and diffusion models, into a single workflow. These functions work on entirely different logics and architectures: Transformers work on sequence data like tokens or musical events, whereas diffusion models generate audio directly from noise. Such a disparity necessitates careful design to ensure meaningful comparison and evaluation of the outputs of these models, considering pre-processing of the data, performance metrics, and evaluation protocols.

Scalability of the System:

Since the project plans to test multiple models on an extensive collection of datasets and styles of music, the scaling up of the infrastructure is a huge concern. The large architecture of the transformers consumes immense computational resources for their training and running inferences on lengthy sequences. Diffusion models due to their refining process through iterations might incur multiple computational steps that would have a greater cost in expanding the system's requirements for scalability. It is really challenging to scale models efficiently for high-resolution audio and for complex data sets.

1.4 PROJECT STATEMENT

Project Symphony of Algorithms: Exploring Effectiveness of Six High-End Deep Learning Models to Generate Original Music Compositions GANs. GRUs. Variational Autoencoders. OpenAI's Jukebox. Transformers. Diffusion Models.

Each model is different and implies a unique approach to the creative process, and for GANs - it is adversarial structure based on the refinement of output between the generator -discriminator collaboration and aiming to create music as realistic as possible.

GRUs - Temporal dependencies Recurrent neural networks that capture rhythmic consistency and coherent compositions. It is the creative diversity in VAEs with the introduction of probabilistic modeling, capable of producing almost a significant variety of musical ideas while keeping structure intact. OpenAI uses Jukebox to develop high-resolution multi-instrument composition which includes vocals - transformer architecture.

Transformers, well known to incorporate attention mechanisms, can capture long-range dependencies and therefore allow for complex musical structures and coherence in the generated compositions.

Class of diffusion models recently introduced: from noise, through refinement iterations to structured music. The paper assesses the models along crucial dimensions to evaluate their capacity for musically rich and innovative composing performances. By integrating computational analysis with subjective human evaluation, the project aims to comprehensively outline strengths and weaknesses of each model in generating diverse musical outputs. The intention here is to examine how well these models are in terms of capturing the intricacies of musical structures, such as harmony, melody, rhythm, and timbre, and then take it forward into indicating possible applications across varied music genres. The comparative analysis will thus provide valuable insights into models that best suit practical use in music production as guides for the development of AI-driven tools for musicians, composers, and producers. The field of AI music composition will emerge as a corollary of the research and inform the future integration of AI into creative industries better.

1.5 OBJECTIVES

Music Composition with Deep Learning Models Investigate how six advanced deep learning models, GANs, GRUs, VAEs, OpenAI's Jukebox, Transformers, and Diffusion Models perform in relation to generating original music compositions. The project will test how each model is able to generate music that is rich in harmonies and rhythmic, novel and coherent with objective measures such as harmonic measures, consistency in rhythms, novelty but also by taking subjective human evaluations into account regarding any type of emotional engagement, musicality, and expressiveness.

Adding Transformers because they demonstrated to capture long-range dependencies in sequential data Diffusion Models that progressively denoise data to generate it, it could make for wider exploration on how different architectures tackle music composition. This project compares these models with more conventional approaches like GANs, GRUs, VAEs, and OpenAI's Jukebox to uncover new understandings of the strengths and weaknesses of each architecture related to creativity, structural coherence, and musical depth.

The project identifies the model(s) that best balances creativity with structural integrity and then explains its possible applications on diverse forms of music. It further elaborates on newer architectures such as Transformers and Diffusion Models to innovate in music generation. The implications would be related to the improvement of AI in creative areas, especially concerning music composition, and provide recommendations for deep learning techniques' improvement to better serve their artistic and cultural applications.

1.6 SCOPE OF THE PROJECT

The scope of this project encompasses the evaluation and comparison of four distinct deep learning models—Generative Adversarial Networks (GANs), Gated Recurrent Units (GRUs), Variational Autoencoders (VAEs), and OpenAI’s Jukebox, Transformers and Diffusion models—in the context of AI-driven music composition. The project focuses on understanding how each model generates original music, how it handles various musical elements such as melody, harmony, rhythm, and structure, and how effectively it balances creativity with coherence.

The project will involve training each model on a diverse dataset of musical compositions across multiple genres (e.g., classical, jazz, pop, electronic) to evaluate their versatility and adaptability in producing music that aligns with the characteristics of each genre. Music generated by these models will be assessed using a combination of objective computational metrics—such as harmonicity, rhythmic consistency, novelty, and coherence—and subjective human evaluations, focusing on musicality, emotional impact, and listener engagement.

Additionally, the scope includes an exploration of the challenges and limitations of each model, such as overfitting, interpretability, and scalability, as well as an investigation into the potential for these models to be used in real-world music composition, production, or experimentation. The project will also address ethical considerations, including issues related to copyright, creativity, and the use of AI in creative fields.

Ultimately, the goal is to provide a comprehensive analysis of AI music generation, offering insights into the potential applications of these models in both commercial and experimental music creation. This project will not only contribute to the development of AI music generation systems but also provide recommendations for future improvements in model design, training techniques, and evaluation methodologies to enhance AI’s role in the music industry.

Chapter 2

Model Architecture

2.1 LITERATURE REVIEW

PAPER 1: MuseGAN: Multi-track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment: <https://arxiv.org/pdf/1709.06298>

In recent years, generative models have gained significant attention for their ability to create complex and realistic data, including in the domain of music generation. Traditional music generation techniques largely relied on rule-based or Markov chain methods, which could produce limited and relatively predictable outputs. The rise of deep learning has led to the application of models like Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks for music generation, particularly for sequence-based tasks like melody generation. These methods, while improving the expressiveness of the generated music, still faced challenges in maintaining structure and long-term dependencies, especially in multi-track or polyphonic contexts.

The advent of Generative Adversarial Networks (GANs) brought a paradigm shift, with GANs proving to be effective for generating high-quality data in various domains. In the context of music, GANs have shown promise in producing realistic music sequences, but challenges remained when extending these models to multi-track music generation. Prior research on music generation with GANs, such as *MidiNet* (Chou et al., 2017) and *MelGAN* (Kumar et al., 2019), focused primarily on single-track or monophonic outputs, often with little focus on the interactions between different tracks in a musical composition. To address these limitations, the paper introduces MuseGAN, a novel framework that extends GANs for the generation of multi-track symbolic music. MuseGAN combines the power of GANs with a multi-track, sequential architecture that allows it to model interactions between different musical tracks—such as melody, harmony, and accompaniment—over time. This architecture also incorporates a multi-scale approach to capture both local and global dependencies within the music sequence. The framework is designed to generate realistic music compositions, and it can also be used for the accompaniment of existing music, which opens up new possibilities for interactive music generation.

MuseGAN also draws inspiration from prior work in music synthesis, including Variational Autoencoders (VAEs) for music generation, and combines them with adversarial training for improved fidelity. The paper demonstrates that the multi-track and sequential nature of MuseGAN provides superior performance compared to previous models in terms of musical coherence and realism.

Music generation has long been a fascinating area of exploration, combining creativity with technology to produce novel compositions. With the advent of deep learning techniques, particularly Variational Autoencoders (VAEs), there has been a surge of interest in leveraging these tools for creative endeavors like music composition. This paper delves into the realm of generating music using VAEs, aiming to contribute to the growing body of research at the intersection of artificial intelligence and music. By harnessing the latent spaces learned by VAEs, we aim to create a system capable of producing diverse and coherent musical sequences while preserving the stylistic attributes inherent in the training data. This introduction sets the stage for our exploration into the application of VAEs in music generation, highlighting the potential for innovation and creativity in this domain.

The intersection of deep learning and music generation has sparked significant interest, leading to the exploration of various approaches within artificial intelligence for creative composition. Variational Autoencoders (VAEs) stand out as a prominent avenue, as evidenced by studies such as [1] and [2], which highlight their ability to capture complex data distributions and generate diverse musical sequences while preserving stylistic attributes. Moreover, beyond VAEs, other deep learning architectures have been investigated, including recurrent neural networks (RNNs) for polyphonic music generation ([3]) and Generative Adversarial Networks (GANs) for realistic musical output ([4]). Recent research has further broadened the scope of AI-driven music generation, with studies like [5] exploring music conditioned on visual inputs and [6] introducing innovative systems for music-conditioned 3D dance generation. Collectively, this diverse literature underscores the ongoing exploration of AI as a tool for artistic expression, encompassing VAEs, RNNs, GANs, and multimodal frameworks, and paving the way for future advancements in creative computational systems.

The outcomes of this research endeavor manifest in a diverse array of generated musical sequences crafted through Variational Autoencoders (VAEs), recurrent neural networks (RNNs), Generative Adversarial Networks (GANs), and potentially other architectures, each evaluated quantitatively for diversity, coherence, and stylistic fidelity, alongside qualitative human assessments offering subjective perspectives on their quality and artistic merit. This comprehensive evaluation framework provides insights into the efficacy of deep learning methodologies in capturing and reproducing musical characteristics, contributing to the advancement of AI-driven music generation. These outcomes not only deepen academic understanding but also have practical implications, guiding the refinement of algorithms and methodologies for creative composition. By bridging technology and the arts, this research stimulates innovation, fostering new frontiers in computational creativity and musical expression.

In conclusion, the exploration of music generation using variational autoencoders (VAEs) highlights the potential of this approach in creating diverse and musically coherent compositions. Despite the challenges in encoding musical information and evaluating the quality of generated music, VAEs offer a flexible framework for capturing the complex structure of musical data and generating novel compositions.

PAPER 3: Jukebox - A Generative Model for Music: <https://arxiv.org/pdf/2005.00341>

The paper "*Jukebox: A Generative Model for Music*" by OpenAI presents a groundbreaking approach to music generation, introducing a model capable of producing high-quality raw audio across a variety of genres, including both instrumental and vocal tracks. Jukebox's contributions build upon existing research in generative modeling, raw audio synthesis, and multimodal learning, combining advances from multiple fields to create a model that generates music directly in the raw audio domain.

Another line of research explored the generation of raw audio using Generative Adversarial Networks (GANs) and autoregressive models. For instance, Donahue et al. (2018) demonstrated the potential of GANs for generating raw audio but faced limitations in quality and sample diversity. Similarly, WaveNet (Van den Oord et al., 2016), a convolutional neural network (CNN)-based autoregressive model, achieved impressive results in speech synthesis and audio generation. However, WaveNet's sequential nature made it difficult to scale to longer audio sequences, and it struggled with efficiency in generating high-fidelity, long-form music.

Jukebox's approach is heavily inspired by VQ-VAE (Vector Quantized Variational Autoencoder), introduced by van den Oord et al. (2017), which allows the efficient learning of discrete latent variables from high-dimensional data like audio. This approach contrasts with traditional autoregressive models by leveraging quantized latent codes to compress the audio, significantly reducing the computational cost while maintaining quality. VQ-VAE has been widely used in other domains, including speech synthesis and image generation, but its application to music synthesis represents a novel contribution.

Another important aspect of Jukebox's novelty is its ability to generate music conditioned on multiple factors, such as genre, artist, and lyrics. This approach is informed by research in multimodal learning where models are trained to understand and generate data that incorporates different types of input. Previous work in text-to-speech (TTS) and music generation has shown that conditioning models on high-level information can lead to more coherent and structured outputs. Jukebox takes this further by conditioning both on genre and artist identity, as well as generating realistic lyrics in sync with music, a challenging aspect of previous models that generated music or vocals independently.

Jukebox is positioned as a state-of-the-art model for music generation that extends prior work in symbolic music generation, raw audio synthesis, and multimodal conditioning. By leveraging VQ-VAE, hierarchical latent spaces, and multimodal conditioning, Jukebox is able to generate diverse, high-quality musical compositions that push the boundaries of what is possible in AI-driven music generation. Future work will likely focus on improving the model's efficiency and expanding its capabilities in vocal synthesis and genre-specific generation.

PAPER 4: MUSIC GENERATION USING RNN-LSTM WITH GRU :

https://www.researchgate.net/publication/375671182_Music_Generation_using_RNN-LSTM_with_GRU

Music generation using deep learning has become an increasingly popular research area due to its potential applications in the music industry, such as automated music composition, personalization of music recommendations, and real-time music generation. Previous studies have explored different deep learning architectures for music generation, including RNN, LSTM and GRU networks.

In a study by Eck and Schmidhuber (2002), they used a bi-directional LSTM network to generate music. By training the model on a dataset of MIDI files, it achieved the ability to generate coherent melodies. The model's exposure to a diverse range of musical patterns and structures within the dataset empowered it to produce melodies that flow harmoniously and exhibit a sense of coherence. However, the generated music lacked the long-term coherence and structure found in human-composed music.

A more recent study by Cheng-Zhi Anna Huang and others. (2018) proposed a framework called Music Transformer, which uses self-attention mechanisms and convolutional neural networks (CNN) to generate music. The model was able to generate music that exhibited long-term coherence and structure, and outperformed previous state-of-the-art models in terms of musical quality.

In a study by Lewandowski et al. (2012), they used an RNN model to generate polyphonic music. Through training on a dataset of symbolic music representations, the model gained the remarkable capacity to generate music that possesses striking similarities to compositions crafted by humans. The model's ability to capture the essence and style of human-composed music adds to its proficiency in producing outputs that resonate with listeners.

Another study by Daniel D. Johnson. proposed a method for generating polyphonic music using an RNN model. The model was trained on MIDI files and achieved the ability to generate music that showcased similar style and structure to the original dataset.

Overall, previous studies have demonstrated the potential of deep learning methods, particularly RNN, LSTM, and GRU, for music generation. However, there is still room for improvement in terms of generating music that exhibits the long-term coherence and structure found in human-composed music. Future research may explore new architectures and techniques to improve the quality and coherence of generated music.

Applications of GANs in Creative Domains Semantic picture editing, style transfer, image synthesis , image super-resolution, and categorization are just a few of the many uses for GANs [6]. They have also demonstrated notable increases in power density and efficiency when applied to power devices including battery chargers, high-voltage DC/DC converters, and front-end power factor correction. GANs have been very effective in the space of picture generation, applying in video, image-to-image, text-to-image, and image synthesis. In addition, GANs have been used in various applications ranging from banking and marketing, fashion design, sports, music, protein engineering, astronomical data processing, remote sensing picture dehazing, and crystal structure synthesis. As far as creative domains are concerned, here are some applications of GANs:

1) Generate Realistic Photographs

Having applications in so many different domains, GANs have proven to have a lot of potential in image creation is a detailed overview of GANs in image creation, focusing on how they can enhance classifier accuracy and data balance. Introduces LR-GAN, the model that respects scene structure and context to create photorealistic images gives an optimized image-generating technique based on Deep Convolutional GAN, generating photorealistic images at high resolutions discusses GAN model performance for image generation and classification issues which found that GAN with convolution layers are more efficient than others in regimes. Together, this research indicate the prospects of GANs in photo production: especially in the context of photorealism and image quality.

2) Image-to-Image Translations

New developments on GAN-based image-to-image translation led to the inception of various artistic models proposed Single GAN, a method that allows an efficient performance for multi-domain image translation using a single generator proposed Selection GAN, a guided model to perform image-to-image translation tasks by imposing semantic guidance to generate high-quality output. InstaGAN proposed by, enhances multi-instance transfiguration by taking advantage of instance data and a context-preserving loss. Demonstrated the role of careful hyperparameter tuning for performance of the model by applying Conditional GANs to image-to image translation. These works, taken together, represent promising techniques in the application of GANs towards picture translation as well as a range of techniques to be explored for their efficiency.

3) Text-to-Image Translation

There are a number of researchers that have applied GANs toward the task of text-to-image translation with offering a Stacked GAN specifically to this end. To generate high-resolution, photorealistic photographs, their method.

4) Face Aging

The application of GANs for facial ageing has been have been explored in various studies. Contextual GANs were proposed by and analyze the changing patterns in the aging faces used an "Identity-Preserving" optimization strategy to emphasize preserving the actual identity in the aged faces. proposed a DCGAN-based model that creates robust and realistic face images by mapping a face to a latent vector and age-conditional vector further enhanced this work by proposing a framework to understand GANs line breakdown of the internal structure of the model, and demonstrate that general-purpose picture sets can be applied to learn the aging transformation. These research studies combined illustrate how GANs can be applied to problems such as face aging, with a focus on identity preservation, transition pattern, and photo-realism in image synthesis.

2.2 MODULES USED

These modules are the essential tools in data handling, machine learning, deep learning, and music processing, among other applications-and very foundational roles in models of music generation similar to OpenAI's Jukebox, VAEs, GANs, and GRUs. Each module, in a nutshell, in relation to these tasks on music generation:

1. Data Handling and Preprocessing

NumPy: The most important library for numerical operations in Python is NumPy, which supports multi-dimensional arrays and matrices, so it's very important when dealing with large datasets, such as those generated to use in music generation. It is used in processing musical features such as spectrograms, pitch classes, or other transformations applied to the data (e.g., normalization).

Pandas: Pandas is useful in manipulating structured data such as metadata of music files- song identifiers, artist names, genre tags, and timestamps-to prepare it for further processing. This is directly relevant in cases with large datasets, especially when referring to the input for training models like Jukebox, which would require preprocessing or organizing information about the music and its labels.

SciPy: SciPy is a package aimed at scientific computing based on NumPy implementing advanced functions for mathematical and statistical application and signal processing. It is applied for the purposes of music processing, such as filtering, Fourier transforms or audio signals analysis.

2. Music Processing

Mido: Music Generation Using Mido, Python Library Handling MIDI files represent musical sequences or compositions Mido reads, writes, and manipulates the MIDI data important for models that generate music in MIDI format; in this pipeline, Mido would be used to convert model outputs into MIDI sequences to play or analyze

Music21: A very powerful toolkit for music analysis and generation, Music21 can support a large number of musical data formats, including MIDI, MusicXML, and ABC notation. It is mainly meant for working with the musical symbols and structures, such as notes, chords, and scales. The use of Music21 is helpful at the preprocessing and postprocessing steps within the generation process of music, such as parsing input music into machine-readable formats or generating musical output from model predictions.

pretty MIDI: PrettyMIDI represents a library for working with MIDI files, with an emphasis on extracting and manipulating features like pitch, note velocity and timing. It's especially helpful for feature extraction when training models (like converting MIDI files to spectrograms or embeddings) and for post-processing music generation models into MIDI format. PrettyMIDI supplies tools for pitch and onset detection, encoding of note sequences, and generation of time-frequency representations. These are all important when working with music.

3. Machine Learning and Deep Learning

TensorFlow: TensorFlow is a popular open-source deep-learning framework used to construct and train neural networks. For example, this package is used to develop complex models for music generation such as VAEs and GANs, and it also implements recurrent neural networks such as GRUs. There are model training techniques like backpropagation, gradient descent, etc., optimization, and deployment. For example, probably using TensorFlow or related deep learning libraries, while handling the massive dataset and training deep neural networks for audio data, OpenAI's Jukebox uses.

PyTorch: Usage for Music Generation: PyTorch is one of the most powerful deep learning frameworks. PyTorch finds the acclamation for its dynamic computation graph, and it is user-friendly and widely utilized in research for experimentation. PyTorch is very commonly utilized in the construction of VAEs, GANs, and even of GRU-based models for music generation. It enables faster prototyping and streamlined application of music-generating models-for example, constructing a custom neural network or applying complex architectures for audio generation.

4. Visualization

Matplotlib: There is use in music generation, as well. Matplotlib remains one of the most popular data visualization libraries in Python; it is largely used for checking the training process (loss curves and accuracy graphs), the outputs of the model (spectrograms of audio samples), or any other metrics (confusion matrices, feature distributions). Audible data or produced music can be visualized as time-frequency plots like spectrograms or note frequency plots to judge how well a model performs and how it behaves.

Seaborn: Seaborn is implemented on top of Matplotlib. It provides a high-level interface to draw attractive statistical graphics. The use cases include exploratory data analysis (EDA), distribution of features, comparison of model performance-comparison of the profile of activation maps in a deep network or comparison of training loss and validation accuracy.

5. Evaluation and Metrics

Scikit-learn: Use in Music Generation. Scikit-learn is a highly recognized library to support machine learning models with tools for evaluations, feature selection, and statistical analysis. The usage of the Scikit-learn model in music generation is not explicit but could be included in applications such as classification, clustering, or regression, where the performance of evaluation in models for machine learning applies. For example, after generating some music from a model, Scikit-learn could be used to compute evaluation metrics like precision, recall and F1-score, or compare the output of a model with ground truth labels for specific tasks, such as genre classification or emotional mood.

2.3 ROLE OF MODULES IN MUSIC GENERATION MODELS

OpenAI Jukebox: This is a deep learning model for the generation of raw audio that applies the concept of convolutional neural networks (CNNs), transformers, and VAEs.

- TensorFlow/PyTorch: These are called upon in creating and training the deep neural network architectures.
- Mido/pretty MIDI: These are called upon in handling MIDI data in terms of pre- and post-processing audio.
- NumPy/Pandas: This would apply for manipulating data and the large datasets found in music.
- Matplotlib/Seaborn: These are used for creating spectrograms, training curves, and evaluating the model outputs.

VAEs: VAEs are generative models which learn a probabilistic distribution over data. For music generation, they can be applied to generate a sequence of notes or audio samples.

- TensorFlow/PyTorch: Encoder-decoder architecture for training a VAE
- pretty MIDI/Music21: To represent the music in a form favorable to training and generating outputs
- NumPy/Scikit-learn: Useful to manipulate the data regarding latent space representations and reconstruction quality.

Generative Adversarial Networks (GANs): Applying GANs to generate new data in a game-theoretic setup between two networks, namely the generator and the discriminator

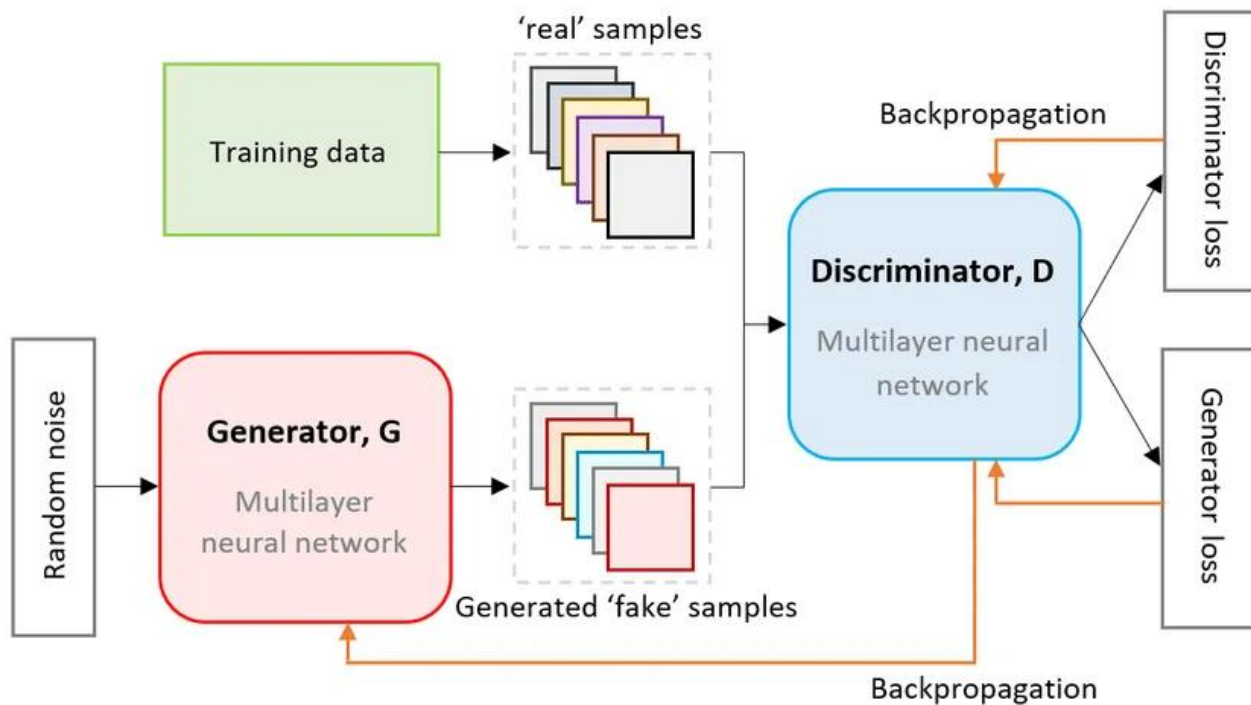
- TensorFlow/PyTorch: The generator and discriminator networks' implementation and training.
- pretty MIDI/Mido: Converted generated music into an analyzable or playable MIDI file.
- Matplotlib/Seaborn: Visualization of the generated samples, training and evaluation metrics.

Gated Recurrent Units (GRUs): GRUs are one of the RNN models widely used in sequence modeling, like musical sequence generation.

- TensorFlow/PyTorch is used to implement a GRU-based architecture, and train the model with musical data.
- Music21/prettyMIDI for working with music representation, and processing output sequences.
- NumPy/Scikit-learn: used for data preprocessing and further assessment of model performance on generated sequences in terms of quality or time-series predictions

2.4 MODELS

2.4.1 GENERATIVE ADVERSERIAL NETWORKS (GAN) ARCHITECHTURE



It follows a basic setup that is typical for GANs, consisting of two main components:

Generator:

The generator is an RNN-based model constructed using LSTM layers with the capability of generating music sequences. This model accepts latent vector input and learns to generate note sequences that resemble those in the training set-Pokemon MIDI music files.

Input Layer: It has a dense layer of size $(\text{latent_dim} * 4)$ followed by a Reshape layer that reshapes the output to a sequence of length 100 with 4 units per time step.

LSTM Layers: There are two LSTM layers, namely, with 128 and 256 units, respectively. These layers learn to discover some temporal dependencies in musical sequences.

TimeDistributed Layers: This is so with TimeDistributed Dense that applies a dense operation at each time step of the sequence and outputs a vector for every time step .

Output Layer: The final Dense layer outputs one value per time step representing a MIDI note that uses tanh activation to generate values between -1 and 1.

Discriminator:

The discriminator, therefore, is a binary classifier that recognizes between real music sequences from the dataset and fake ones generated by the generator.

It runs the sequence input through the LSTM layers to pick up the temporal dependencies of the music:

The first LSTM layer has 256 units and is using a return sequence, as it should provide the output for the following LSTM layer.

The second LSTM layer uses 128 units followed by a Dense layer with 64 units to make further classification decisions

Lastly, the Dense layer makes a prediction value on a probability scale from 0 to 1 as a sigmoid activation to classify whether the input sequence is real or fake.

GAN (Generative Adversarial Network): The GAN is composed of the generator and discriminator models. While training, the discriminator is trained on how to identify real versus fake sequences; the generator is trained on the job of fooling the discriminator.

The input to the generator is the random latent vector, and this generates a sequence that is sent to the discriminator to predict whether it is real or fake.

Both components use a binary cross entropy loss function and are optimized using Adam.

2.4.1.1 PREPROCESSING

MIDI File Extraction: The list of Pokémon MIDI files is read from a ZIP archive. It is saved in a folder for further treatment.

MIDI to Note Sequence Conversion: Every MIDI file is imported from the mido library and read to get note sequences. A note sequence is formed by iterating over all tracks of the MIDI and pulling the note on events where a note was played and a velocity is greater than zero. The values of the extracted notes are added to the sequence for later use in the training of the model.

Padding/Truncating Sequences: To make uniformity into the input data, the note sequences are truncated or padded to a constant length of 100. All sequences with more than 100 notes are truncated while those with less than 100 are padded to a length of 100 using zeros.

2.4.1.2 METHODOLOGY

In general, the methodology of the project seems to focus on training a GAN to produce sequences of realistic music, and that can be in the form of file progressions within MIDI format based on the Pokemon dataset.

GAN Setup: The generator is trained to generate fake sequences from random noise or latent vectors. A discriminator is trained to differentiate between real and fake sequences. It essentially classifies the real data-set sequences as "real" and the ones generated by the generator as "fake".

Adversarial Training: During training, the generator and discriminator get updated in an alternating manner. The discriminator is initially trained on a batch of real and fake sequences. It then trains the generator by updating the generator on better fake sequences so as to fool the discriminator.

Loss Functions: The discriminator and generator both use binary cross-entropy loss so as to measure how well they perform. What the discriminator wants to do is to classify real and fake sequences correctly, while what the generator wants to do is to minimize its ability to distinguish between real and fake sequences.

2.4.1.3 EVALUATION METRICS

To assess the performance and diversity of the generated sequences, several metrics are used:

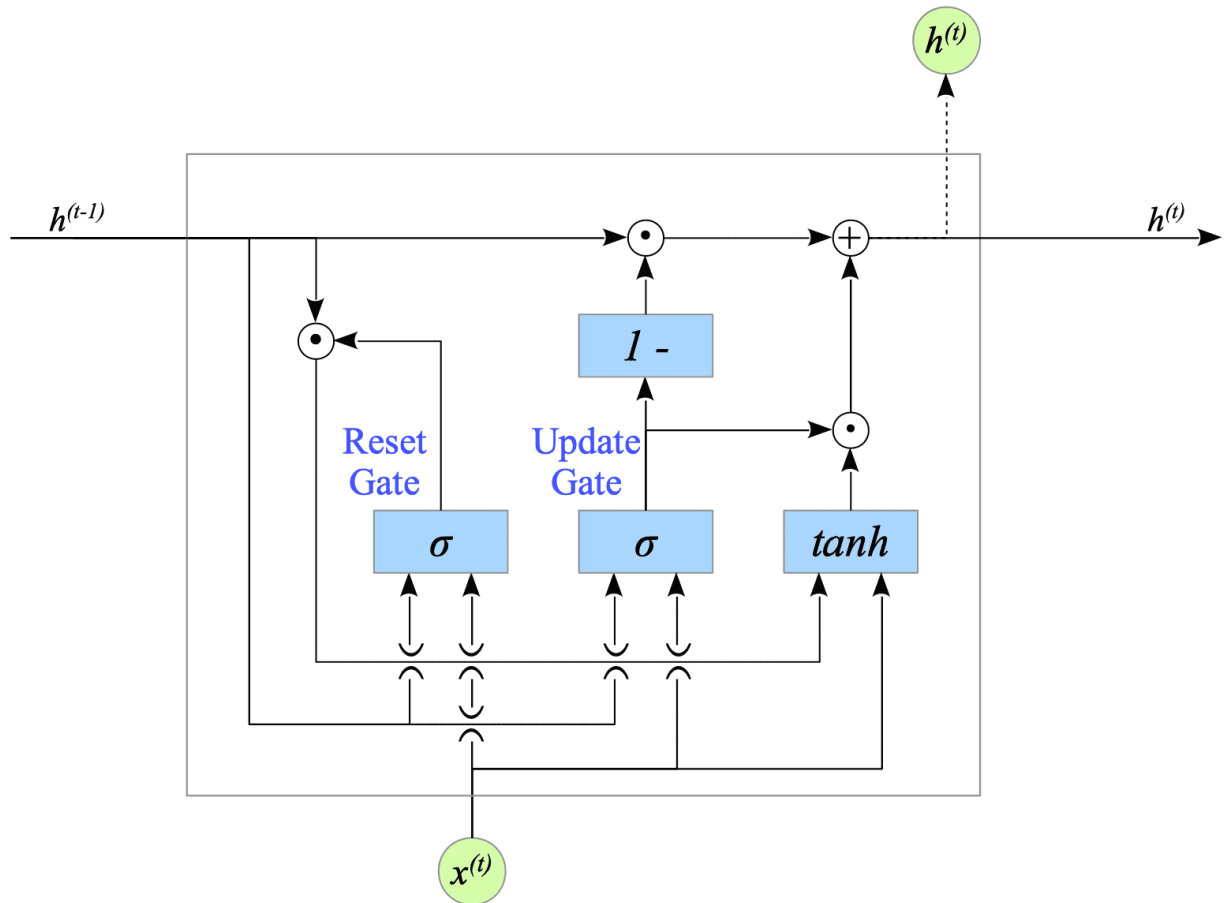
1. **Diversity Metric:**
 - This metric evaluates how diverse the generated sequences are. The diversity score is calculated as the average Euclidean distance between each pair of generated sequences. A higher diversity score indicates that the generated samples are more distinct from each other.
2. **Novelty Metric:**
 - This metric measures how different the generated sequences are from the real sequences in the training data. It computes the Euclidean distance between each generated sequence and the closest real sequence in the dataset. The novelty score represents how "new" the generated sequences are compared to the real data.
3. **FAD Proxy Metric (Fréchet Audio Distance Proxy):**
 - This metric compares the generated sequences to the real sequences in terms of both mean and variance. The mean squared error is calculated between the means and variances of the generated and real sequences. A lower FAD score indicates that the generated sequences have similar statistical properties (mean and variance) to the real sequences.

2.4.1.4 MODEL SUMMARY

The model described in the paper is a GAN that generates musical sequences, in this case, through the form of MIDI files, Pokémon-specific music. It has two major constituent parts, namely the Generator and the Discriminator. There's a brief overview of each part and overall GAN setup.

Based on a GAN, the model architecture is designed to generate realistic sequences of Pokémon music in a MIDI format. It includes a generator, which has LSTM layers producing sequences from random latent vectors, and a discriminator, also with LSTM layers, classifying the sequences as real or fake. The model is trained using adversarial learning, and the evaluation is achieved by calculating diversity, novelty, and FAD proxy metrics for the quality and originality of the generated music.

2.4.2 GATED RECURRENT UNIT (GRU) ARCHITECTURE



This is a Recurrent Neural Network (RNN) composed of Gated Recurrent Units (GRUs). The model is sequential as in music data represented by MIDI files, where the task is to predict the next note given sequence of preceding notes.

Here's a brief description of each component:

a) **Embedding Layer:** This layer takes in a sequence of input notes and converts each into a dense vector representation. The input dimension is the total number of unique notes, and the output dimension is set at 100. So, every note becomes a 100-dimensional vector.

Why: Embedding layers help transform sparse representations-integer encodings of notes-to dense and more meaningful vector representations. This is very critical in capturing semantic relationships between the notes in the music.

b) GRU Layers (Gated Recurrent Units):

GRU is a derivative of LSTMs, but without the nuances of the gate mechanism. First layer GRU has 256 units and Output at every timestep will be passed to next layer. GRU layers are used in sequence-based problems such as generating music, as they can retain and learn temporal dependencies across sequences. Since it is a recurrent one, it means it can track the pattern of the musical notes over time.

c) Dense Layers: That's fully connected layer with 128 units and ReLU activation.

Why: After the GRU layers have extracted sequential patterns, the dense layer aids in making the final classification by transforming the extracted features into a suitable format for the output layer.

Dropout is a regularization technique that randomly sets a fraction (30% in this case) of input units to 0 during training to prevent overfitting.

d) Output Layer: A final dense layer with SoftMax activation computes the output probabilities for all possible notes. The number of units equals the number of unique notes. The model will have to predict the next note in a sequence. SoftMax ensures that the output is a valid probability distribution over all possible notes, which makes it appropriate for classification tasks where each output corresponds to a note.

2.4.2.1 PREPROCESSING

a) Note Extraction from MIDI Files:

Extracts the note values, i.e., the pitch, from the MIDI tracks. All MIDI files are nothing, but a sequence of events mapped to "note on" messages. MIDI files are very much like text files- the variations in it represent musical information. This would allow the extraction of note values to let the music be fed into a neural network for training purposes.

b) Convert notes into integers: In this section, the code maps each unique note to an integer value. It also creates a reverse mapping to convert the predicted integers back into notes. Since neural networks work with numerical values, the notes (which were originally categorical) are mapped to integers so that the model can process them.

c) Sequence Generation: It creates sequences of fixed-length notes, where `sequence_length = 50`. The input sequence is a list of 50 consecutive notes, and the output would be the note following the sequence. Music generation models require input-output pairs that allow the model to recognize which note is most likely to follow the given sequence.

d) Output One-Hot Encoding: The output notes are one-hot encoded so that each output note is represented as a vector with a 1 at the index corresponding to the note and 0 elsewhere. One-hot encoding is used since it's a classification problem, and the model needs to predict one out of many classes; in this case, it is the next note.

e) Train-Test Split: Splitting data into training and test sets, where 80% of the data is used for training and 20% for testing. It allows evaluation of the model on unseen data to check on its generalization ability.

2.4.2.2 METHODOLOGY

This approach is based on a sequence-to-sequence learning approach, in which:

A sequence of notes is input to the model. The model will learn the GRU layers to learn the temporal pattern in the sequence. The model then predicts the next note in sequence to create new music. The training incorporates backpropagation with categorical cross-entropy loss optimized by Adam.

2.4.2.3 EVALUATION METRICS

The major metrics to assess the performance of the model during training are:

a) Loss: Categorical cross-entropy loss function, which is the right function to use for multi-class classification problems such as this where a model needs to predict one note out of many. The loss is a measure of how well the model predictions match the actual data. The lower the loss, the better the model.

b) Accuracy: To calculate the fraction of correct prediction of the model. It simply gives a perception of the good practice a model is doing, giving a percentage of the notes which are predicted correctly by a model compared to the true note.

c) Plots: Training and validation loss and accuracy are plotted in terms of each epoch. These plots will be helpful to figure out whether the model is overfitting or underfitting, or if it is converging properly.

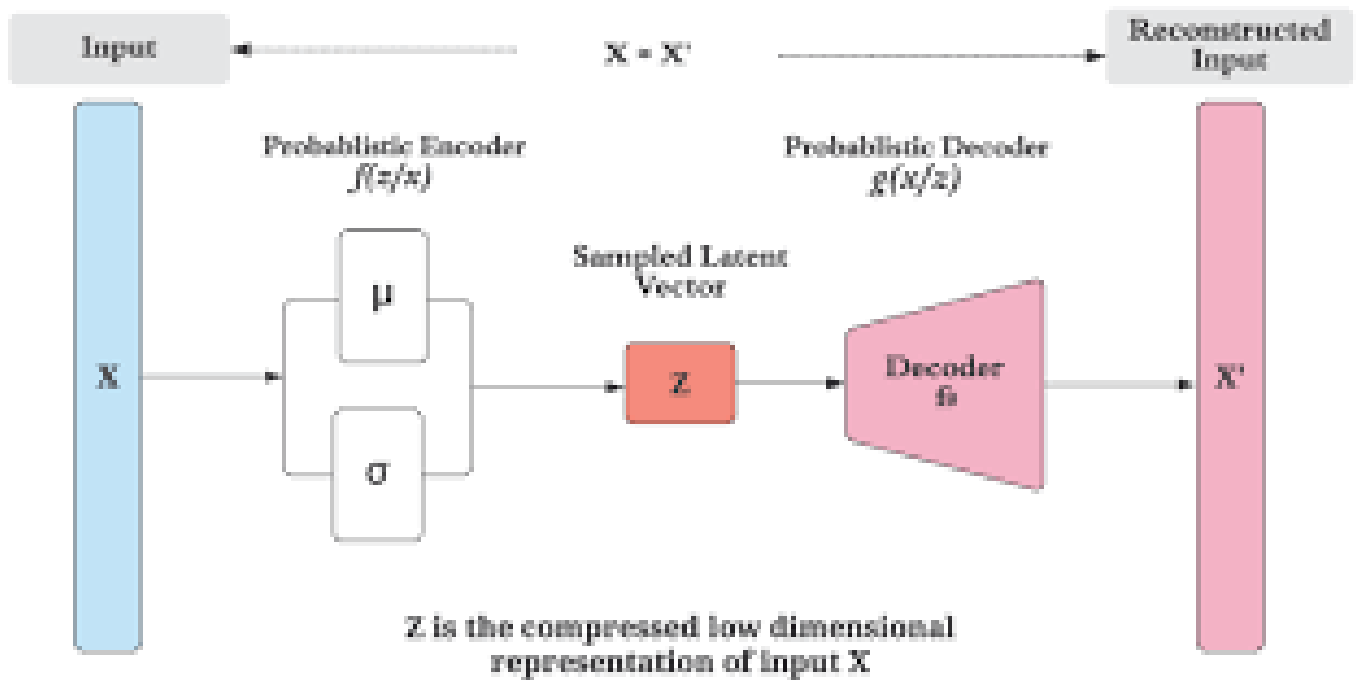
2.4.2.4 MODEL SUMMARY

The model is a sequential neural network designed for music generation using Recurrent Neural Networks with Gated Recurrent Units. This architecture begins with an embedding layer, which maps integer-encoded notes into 100-dimensional dense vectors.

The sequence then passes through two GRU layers: the first has 256 units and outputs the sequences of 50 steps in time; the second has 256 units and produces the final vector of 256 dimensions. Next is a dense layer of 128 units with ReLU activation, and dropout of 30% to avoid overfitting. SoftMax activation is used for the final output layer, that is, the last dense layer. Thus, the model predicts the next note by returning a distribution over all possible notes.

The total number of trainable parameters of this model are 801308.

2.4.3 VARIATIONAL AUTOENCODERS (VAE) ARCHITECHTURE



This model is based on the architecture of an autoencoder that adapts to represent data in a compressed latent space. Here, the VAE comprises an encoder, reparameterization trick, and a decoder:

Encoder: The encoder accepts the input MIDI sequence represented as a flattened vector of normalized MIDI note values and turns it into a compressed latent representation. It includes a linear layer that reduces the input dimensionality, determined by max MIDI sequence length, to a hidden dimension. A ReLU activation layer for nonlinear transformation a last linear layer in order to output mean and the log of variance of latent space distribution, hence the model learns both parts of the distribution.

Reparameterization Trick: This adds the functionality of backpropagation to the sampling mechanism in latent space. Sampling from a standard normal, the model scales it by the learned mean and variance to get a latent variable z to feed to the decoder.

Decoder: The decoder accepts the representation and attempts to recreate the original sequence of MIDI. It consists of

Linear layer that maps latent dimension to the hidden dimension ReLU activation layer.
 Final linear layer with Sigmoid to produce values in the range $[0,1]$ because the MIDI values are normalized.

2.4.3.1 PREPROCESSING

MIDI file loading:

Loading the MIDI files is carried out by employing music21 where for each file within the given directory the note pitch value coming from that file was read and then transformed into a numerical MIDI number for each pitch.

Normalization:

MIDI values are in the range 0-127. So every value will be normalized to the interval [0,1] by dividing by 127. This makes the training of VAEs more stable since the data will lie in small range.

Padding:

Since the time sequences are of varying lengths, we need to padding every MIDI sequence with zeros at the end to equal the longest sequence. This helps ensure consistent input dimensions throughout our dataset.

2.4.3.2 METHODOLOGY

Data preparation: Prepared as a PyTorch dataset with the aid of a custom MidiDataset class that pads each MIDI sequence to uniform length.

Model definition: The VAE model is defined in line with the architecture.

Loss function: VAE's loss function combines:

Binary Cross-Entropy (BCE): The reconstruction loss between the original and reconstructed MIDI sequence.

KL Divergence Kullback-Leibler (KL): Pushes the latent space to be closer to a normal distribution. This will be helpful in making samples from this latent space as realistic.

Training For each epoch of the model: Passes a batch of MIDI sequences from the encoder to the decoder by reparameterization.

Computes the loss: sum of BCE and KL divergence backpropagates the loss to update the model weights. Tracks and prints the metrics for the epoch, including the loss value and accuracy.

Music Generation: After training, the model generates a one-minute music piece by sampling from a latent space and decoding those samples to format them as MIDI notes.

2.4.3.3 EVALUATION METRICS

Total Loss: This would sum up the reconstruction (BCE) loss and KL divergence. Lower values indicate that your model fits better.

BCE Reconstruction Loss: measures how well the model could reconstruct MIDI sequences from their latent representation. Lower values will have a better reconstruction accuracy

KL Divergence Loss: How similar is the latent space with being normal. Lower KL Divergence values indicate a regularized latent space.

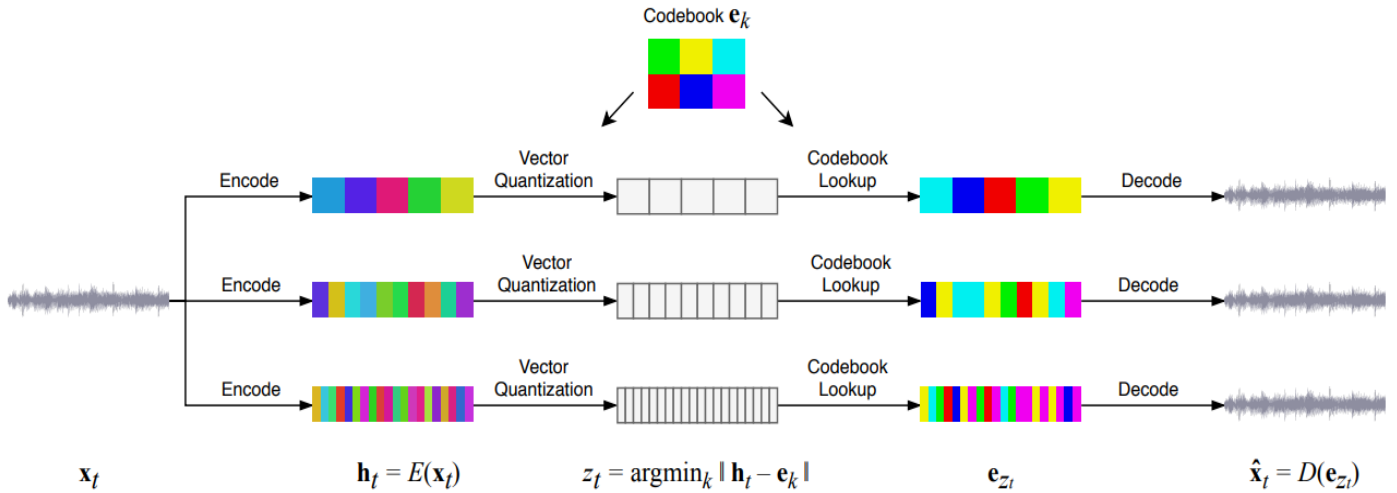
Accuracy is determined by thresholding reconstructed values and then comparing with the original ones, and this provides information on the basis of how similar the reconstructed sequences are to the originals on a binary level.

2.4.3.4 MODEL SUMMARY

The proposed architecture uses a VAE to learn compact, continuous representations of MIDI sequences. At this time, the model is trained to reconstruct MIDI sequences such that the latent space is close to a normal distribution. Sampling from the latent space is then utilized for music generation, making it possible to produce new MIDI music after training. Using reconstruction accuracy and regularization of the latent space, the model can be utilized for testing the performance in generating realistic and coherent sequences of MIDI.

2.4.4 OPENAI JUKEBOX ARCHITECHTURE

Jukebox: A Generative Model for Music



OpenAI Jukebox uses the architecture of VQ-VAE-2 combined with autoregressive modeling to capture long-term dependencies in music. The main components are described below.

VQ-VAE-2 Hierarchical Structure: Jukebox is a hierarchical VQ-VAE-2 model with three levels: top-level, middle-level, and bottom-level encoders and decoders. This structure allows the model to output high-level features at the top level and progressively refine details through the middle and bottom levels. Each layer of the hierarchical structure is recording the music at different resolutions. The highest level will capture the overall form of the music, while the lowest level captures more detailed information such as the actual sound of an instrument .

Discrete Codebook Representations: Using a learned codebook, VQ-VAE-2 encodes audio into discrete codes, which allow for efficient compression and hence enables the model to focus on meaningful representations of the audio.

Autoregressive Transformer Model: These discrete tokens are then used by an autoregressive Transformer, modeling token dependencies. This then trains the Transformer network to predict the sequence of tokens at every level in the hierarchy and is important for coherent, structured music over time.

Decoder Stack: The model has decoders at each level that reconstruct the audio from the encoded tokens back into a playable audio waveform.

2.4.4.1 PREPROCESSING

Audio Conversion and Sampling:

Convert all the audio files to an appropriate format, either 44.1 kHz or 48 kHz, stereo channels. This kind of standardization is needed in order to feed the data into the VQ-VAE model.

Segmentation and Chunks:

For long audio files, chunks or segments are used instead to make the data more manageable and to enhance the efficiency of the training. Also, the chunks are usually a few seconds long, capturing not only melodic patterns but also rhythmic structures.

Quantization:

Jukebox reduces the quantization of audio into discrete representations by encoding waveforms using VQ-VAE. It turns continuous audio to sequences of discrete tokens that represent chunks of the audio waveform and enables Transformer-based sequence prediction.

Augmentation (if applicable):

These can then be optimized for robustness using transformation, such as pitch shifting, time stretching, or adding noise. Such techniques improve generalization of the model when developing a much larger variety of audio outputs.

2.4.4.2 METHODOLOGY

Hierarchical Training:

Jukebox trains each level of the VQ-VAE-2 hierarchy sequentially: the top-level encoder-decoder is trained first, followed by the middle one and finally the bottom.

This training scheme allows the model to progressively learn to capture long-term structure before focusing on finer details.

Transformer-based Sequence Modeling:

Then, the output tokens of each stage of the VQ-VAE are used to train a time sequence autoregressive Transformer which, in turn, trains over time dependencies conveying rhythmic and melodic progressions.

Conditional Generation (with labels):

If there are metadata or labels available - in this instance, genre, mood, or perhaps even specific themes of Pokémon-then the model can be conditioned on those inputs to really control the style and theme of the produced music.

2.4.4.3 EVALUATION METRICS

The evaluation of the music generation model is subjective. Many quantitative and qualitative measures are proposed in the literature to measure the performance of a model. A few such are mentioned below.

Spectral Convergence:

This measures how close the generated audio spectrogram is to the spectrogram of the real music data. The higher spectral convergence implies better audio fidelity.

Mel-frequency Cepstral Coefficients (MFCCs):

MFCC similarities between generated audio and real audio data are compared. A smaller distance in MFCCs indicates that the generated music is closer to the original training data.

Inception Score (For Music):

Similar to the use of inception scores for image generation, one can adapt an inception score to measure just how diverse and recognizable the generated music is compared to the Pokémon themes.

User-based Evaluation:

Rating the generated music by human listeners in terms of coherence, thematic relevance and overall musical quality is normally subjective in nature but can be made more objective with regards to a Pokémon-themed dataset by asking the evaluator if the music is coherent with anticipated character themes or moods.

Accuracy of Genre and Style Classification

If conditioned on genre or style, a classifier should be able to tell whether the generated music is in the intended theme or genre, so this will assist in assessing how well the model conditions upon the inputs.

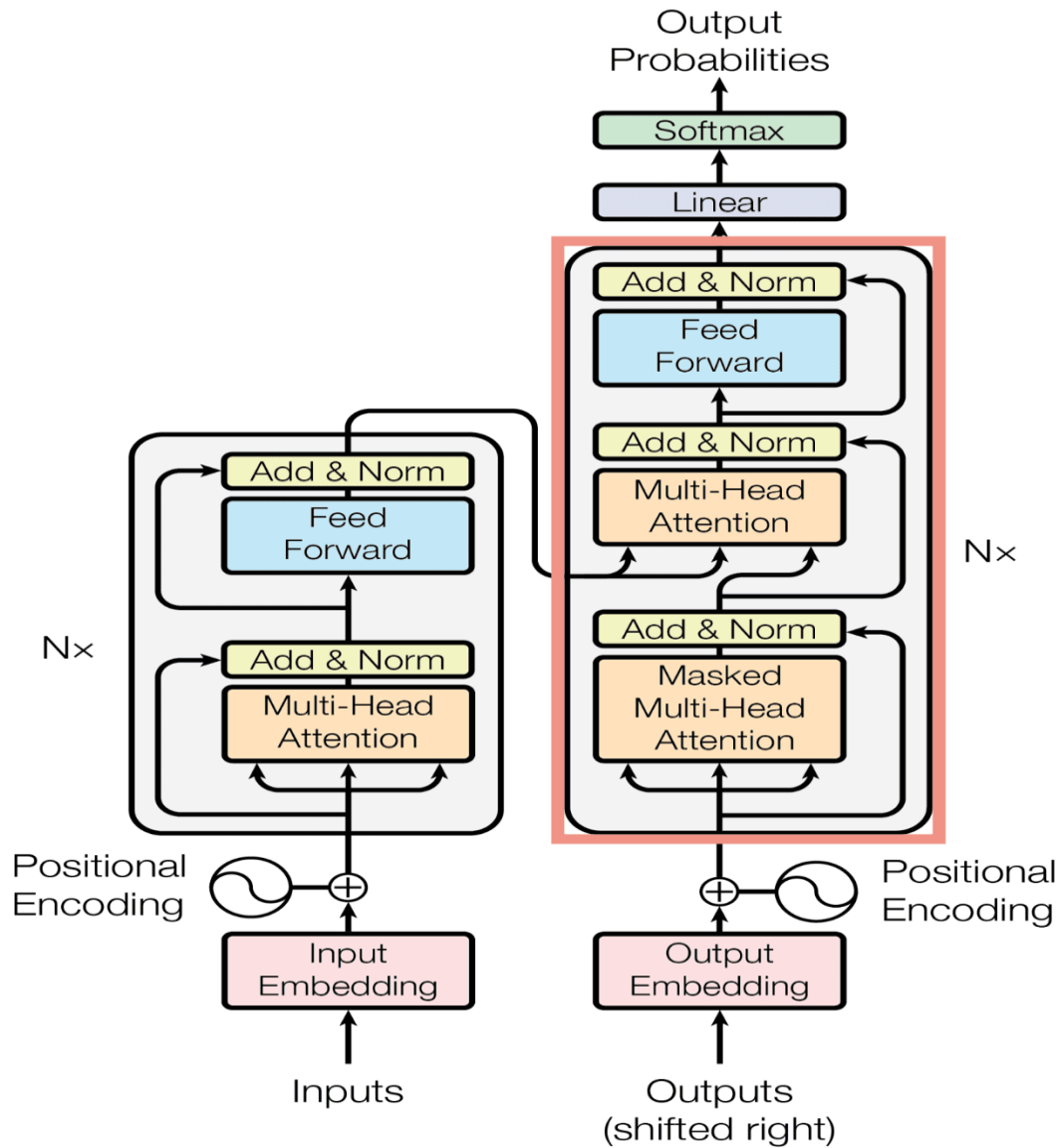
2.4.4.3 MODEL SUMMARY

OpenAI Jukebox is a deep learning model which produces high quality music using a hierarchical VQ-VAE-2 architecture combined with an autoregressive Transformer. It encodes audio at multiple resolutions to capture long-term structure and fine detail within the audio.

The model can be conditioned on attributes such as genre or mood to generate music specific to that theme. The evaluation metrics used are spectral convergence, MFCC similarity, and human ratings for coherence and quality.

One of the real potent models in the deep learning paradigm that can generate good high-quality, diverse music across quite divergent genres and styles is the Jukebox. It works through a hierarchical mechanism, where VQ-VAE-2 can be applied for efficient encoding of audio, and it pairs with autoregressive Transformers to allow long-term modeling of sequence information, making Jukebox able to create coherent and detailed music with its conditional generation. It can be applied to generate themes at specific times, hence applying it in genre-specific music generation or theme-based composition.

2.4.5 TRANSFORMERS ARCHITECHTURE



The model architecture taken is GPT-2, which is a transformer-based model with adaptation towards using it for autoregressive generation. Transformers are widely adopted for sequential data-related tasks such as language and music generation, as they are considered very effective in learning long-range dependencies.

Transformers rely on self-attention mechanisms whereby every note in a sequence can attend to every other note in the sequence enabling smooth temporal patterns in a model.

Layers and Attention:

GPT-2 consists of various layers of self-attention. In every layer, multiple attention heads are present. It thus allows every head to attend to different parts of the input sequence, thereby allowing the model to learn complex patterns in the MIDI note sequences.

In this case, each input is a sequence of tokenized notes, where every note is represented by an index in the vocabulary derived from the dataset.

Output Layer:

The final layer is a linear transformation mapping the hidden representations to a probability distribution over the vocabulary of notes. The model will output a sequence of tokens, each token representing the next note in the sequence, given as input the previous notes.

2.4.5.1 PREPROCESSING

MIDI to Note Sequences:

The MIDI files contain information related to musical notes. They include note-on and note-off messages, velocity, and timing. The sequence of note-on events was extracted from each file. For each track of every MIDI file only note-on events with non-zero velocities are taken as those refer to played notes in the music.

Tokenization:

All the notes were collected and a vocabulary was obtained by assigning an unique index to each distinct note. Each note sequence was transformed into a sequence of integers, where each integer represents a unique note across the dataset.

Padding and Truncation:

Sequences were padded or truncated to a given fixed length (sequence length = 50). Short sequences were padded with zeros, ensuring all sequences are of equal length.

Only the first 50 were retained for sequences longer than 50 notes.

Dataset Split:

The processed dataset was divided to enable model training on and evaluation using separate data. The division was at 80 - 20 percentages of training and validation sets.

2.4.5.2 METHODOLOGY

Training:

This model was fine-tuned with the Pokémon MIDI dataset through note sequences. The input to the model is a sequence of notes, and the model is trained to predict the next note in the sequence. Cross-entropy loss was used for training since it is standard with classification and language modelling tasks.

Sequence Generation:

After training the model, we allowed it to generate new sequences of notes in which the model predicts one note at a time and feeds that back as the input. This process was repeated several times till we got a predefined sequence length of, say, 100 notes.

Temperature Sampling:

A temperature parameter controlled the randomness of predictions during generation. Lower values made the model more confident and conservative; higher values allowed for much more randomness, with this model becoming highly creative but probably less coherent.

MIDI Generation:

The generated sequences of the note were then converted back to MIDI format. For every note index, a MIDI note value was mapped appropriately into a MIDI file that could be replayed as music.

2.4.5.3 EVALUATION METRICS

Loss (Cross-Entropy Loss):

The cross-entropy loss is the first metric to be measured to assess loss, which computes how well confident a model is with the predictions of its notes. The lower the loss, the better its performance is on next-note accuracy prediction.

Accuracy:

This was computed by counting the match between the correctly predicted note and the actual note at each sequence. This is the percentage of correct notes being predicted by the model.

Perplexity :

Perplexity is the exponentiated variant of cross-entropy loss, which will provide an estimation of the how well a model predicts a sequence. A lower perplexity means the model has learned meaningful patterns within the music, whereas high perplexity values indicate uncertainty in predictions.

Precision, Recall and F1 Score:

Precision and recall were calculated on the predicted notes versus actual notes. Precision is how often your predictions are correct while recall measures the proportion of true notes you correctly found. The F1 score gives you the harmonic mean of precision and recall. So, it balances the trade-off between

precision and recall.

Confusion Matrix:

A confusion matrix is plotted to show the distribution between the predicted notes and actual notes and to understand, which were mispredicted and potential biases in the model learning.

Confusion Matrix Visualization:

Heatmap of the confusion matrix was plotted to reveal the performance of the model across all the note predictions and to help diagnose specific misclassifications in the note sequences.

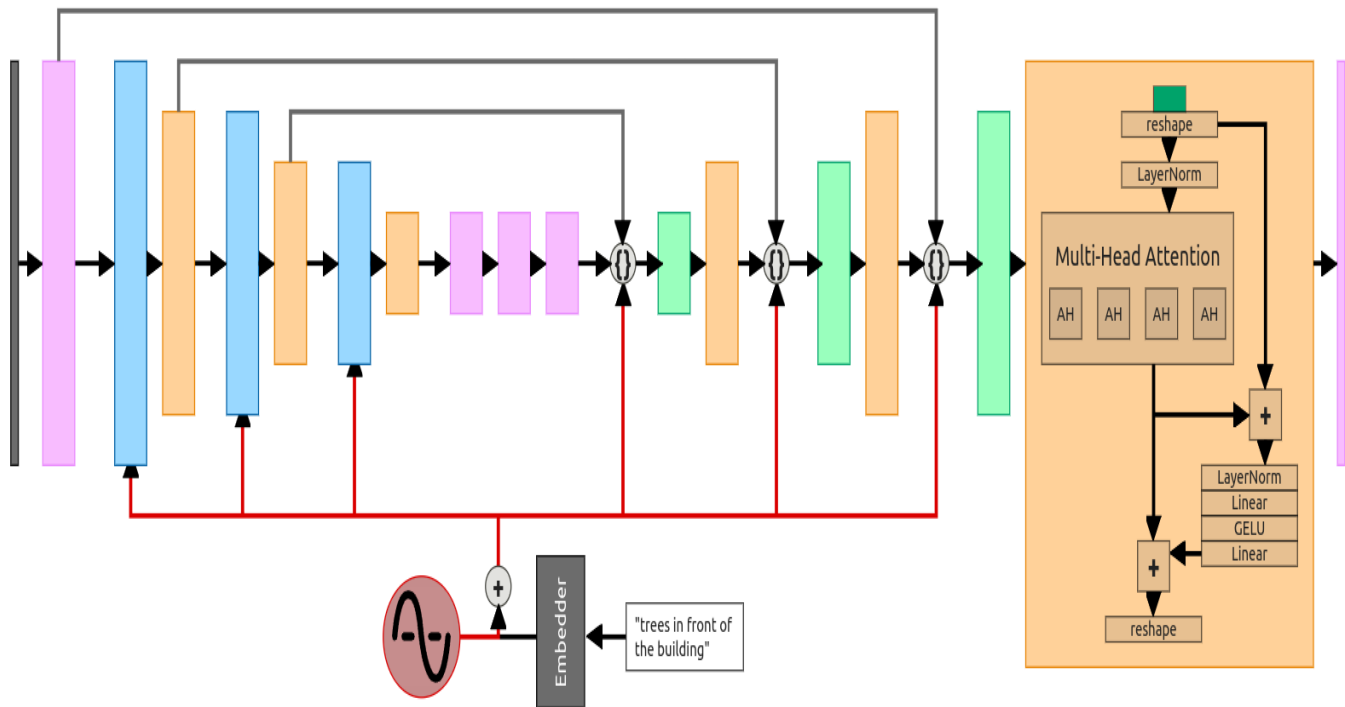
2.4.5.4 MODEL SUMMARY

It uses the GPT-2 model, a transformer-based architecture, to generate sequences of MIDI notes by learning the next note in that sequence, trained on tokenized MIDI data from the Pokémon MIDI dataset. The model goes through numerous layers of self-attention, each with attention heads, able to capture long-range dependencies in music sequences.

It comprises event extraction of note-on events into an integer sequence called tokenized notes, followed by sequence uniform length due to padding or truncation. It trains with cross-entropy loss in order to predict the next note in a sequence and generates sequences of notes using autoregressive sampling with a temperature parameter to modulate creativity.

It is then evaluated with measures such as cross-entropy loss, accuracy, perplexity, as well as precision, recall, and F1-score. A confusion matrix also helps in deducing mispredictions as well as biases in the learning of the model. The final output is the MIDI sequence that can be played as music.

2.4.6 DIFFUSION MODEL ARCHITECTURE



The architecture of the Diffusion Model utilized for this music generation task works with exactly the same principles as generally diffusion models but is so designed to work on sequences of musical notes, namely MIDI data. Let's first break down the core components:

Embedding Layer (self.embedding):

It takes musical notes (which, as integers, correspond to MIDI pitch values), embeds them into dense vectors of a size specified at the beginning. In other words, it instructs the model to represent the notes in a higher dimensional space in which musical patterns and relationships can better be described.

Noise Adder and Noise Remover Layers (self.noise_adder and self.noise_remover):

These are fully connected layers (with the same input/output dimension) that support the important operation of diffusion models: adding noise and later removing it. The `noise_adder` adds noise to the musical notes at hand, which have been represented through an embedding process where the weights scale a random noise. In contrast, the `noise_remover` tries to revert this process in order to retrieve a cleaner version of the sequence.

Decoder Layer (self.decoder):

After it has removed the noise, it feeds the cleaned embeddings into the decoder that leads

to the pushing of these embeddings back to their initial token space; this token space is equivalent to musical notes. This output is a vector of logits over the 128 possible values in MIDI pitches representing the probability distribution for the next note.

Forward Pass:

Forward pass It takes as input the notes to be inputted in the form of a sequence x and scalar time step t . The time step is deciding how much noise to add. The forward pass is composed of:

Implicit representation of input notes

Addition of noise scaled by the time step, over the sequence

Noise removal and recovering the cleaner representation

Decoding the representation into the logits for future notes.

2.4.6.1 PREPROCESSING

Some preprocessing is applied to the MIDI data right before it is fed into the model:

MIDI dataset extraction: This Pokemon MIDIIs.zip dataset is found in a zipped format. The zip is extracted and MIDI files read out to process further.

MIDI to Note Sequence Conversion: It converts each MIDI file into a sequence of note tokens that are pitch values. By using the pretty Midi library, the pitch of each note contained in a MIDI file can be extracted. For simplicity, in this problem, only the pitch - the MIDI note value - is used as the token.

Procedure: This procedure takes in each MIDI file and converts it into a sequence of integer tokens, which represent musical notes.

Padding of Sequences: Since the lengths of different midi files can vary, we pad sequences to have the same batch-size. This padding is done by `torch.nn.functional.pad`; because in our case, all the sequences of a batch are of equal lengths, their shorter sequences are padded with zeros.

2.4.6.2 METHODOLOGY

The basic concept of diffusion models is that input data would be noised gradually, and then reverse the process to recover the original data. The main steps of the methodology are as follows:

Forward Process (Noise Addition):

Random noise added at intervals to the original sequence of notes in the forward process by the time step t . It is actually simulating the corruption of the data by noise itself. Noisier as the t increases.

Reverse Process (Noise removal): The model wants to learn how to undo this forward process — that is, how to denoise the noisy sequence back to the original sequence of notes. It does this by learning a

denoising function, which attempts to remove the added noise at each step.

To accomplish this, the model uses two components:

Noise Adder:

Adds noise to the sequence based on the time step t .

Noise Remover:

Try its best to remove the noise and clean the sequence.

Loss Function:

It trains the model on such loss function that it tries to minimize the reconstruction loss. Reconstruction loss gives a note of how well the model is reconstructing the original sequence from a noisy one. In this case, the loss function that is used here is Cross-Entropy Loss that tries to compare the predicted sequence regarding logits with the true sequence of notes.

At inference-for example, composing music-the model outputs new musical notes by iteratively sampling the output distribution usually with argmax or similar temperature-controlled sampling techniques.

The model predicts one note after another by predicting the next note to complete the desired sequence and repeats.

2.4.6.3 EVALUATION METRICS

Reconstruction Accuracy:

These measures how well the model can reconstruct a sequence of notes when noise is introduced and then removed. Accuracy is computed by comparing the predicted notes with the ground truth notes by count the number of correct predictions.

Perplexity:

Perplexity is the ability of a model to predict the next token in each sequence. Perplexity is computed as an exponential value of the average loss per token. That is, the better the perplexity, the better is predictive performance.

Range Coverage of Pitch:

This metric counts how many unique pitches are used by the generated sequence, for measuring diversity of the generated music. The more unique pitches, the better the model must be able to create more diverse, complex music.

Music Length and Diversity:

It also measures the time length of the generated music and diversity of notes produced. The ability to create long, varied musical sequences is critical for a task like music generation.

2.4.6.4 MODEL SUMMARY

This music generation model uses the principles of diffusion to generate MIDI note sequences through progressive addition and removal of noise. It involves an embedding layer that maps the MIDI pitch values into dense vectors, noise addition and removal layers simulating the diffusion process, and is then trained to learn the denoising of sequences as well as predict a note from its input sequence via a decoder. During training, the model learns to reconstruct noisy input in terms of the original musical notes, with the use of Cross-Entropy to minimize reconstruction loss. The work is also empirically evaluated on some additional metrics, including reconstruction accuracy, perplexity, pitch diversity, and finally on the length and variety of produced music. When inferring, the model generates new music by iteratively predicting one note at a time, composing longer sequences.

3 RESULTS AND METRICS

3.1 GENERATIVE ADVERSERIAL NETWORKS (GAN)

GENERATOR MODEL SUMMARY:



Generator Model Summary:
Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_5 (Dense)	(None, 400)	40400
reshape_1 (Reshape)	(None, 100, 4)	0
lstm_4 (LSTM)	(None, 100, 128)	68096
lstm_5 (LSTM)	(None, 100, 256)	394240
time_distributed_2 (TimeDistributed)	(None, 100, 128)	32896
time_distributed_3 (TimeDistributed)	(None, 100, 1)	129

=====
Total params: 535761 (2.04 MB)
Trainable params: 535761 (2.04 MB)
Non-trainable params: 0 (0.00 Byte)

DISCRIMINATOR MODEL SUMMARY:



Discriminator Model Summary:
Model: "sequential_3"

Layer (type)	Output Shape	Param #
lstm_6 (LSTM)	(None, 100, 256)	264192
lstm_7 (LSTM)	(None, 128)	197120
dense_8 (Dense)	(None, 64)	8256
dense_9 (Dense)	(None, 1)	65

=====
Total params: 469633 (1.79 MB)
Trainable params: 0 (0.00 Byte)
Non-trainable params: 469633 (1.79 MB)

GAN MODEL SUMMARY:



GAN Model Summary:
Model: "model_1"

Layer (type)	Output Shape	Param #
input_2 (InputLayer)	[None, 100]	0
sequential_2 (Sequential)	(None, 100, 1)	535761
sequential_3 (Sequential)	(None, 1)	469633

=====
Total params: 1005394 (3.84 MB)
Trainable params: 535761 (2.04 MB)
Non-trainable params: 469633 (1.79 MB)
=====

DIVERSITY METRIC:

▼ Diversity Metric

Diversity is calculated as the average pairwise distance between generated samples.

```
[ ] def diversity_metric(sequences):  
    n = len(sequences)  
    distances = []  
    for i in range(n):  
        for j in range(i+1, n):  
            distances.append(euclidean(sequences[i].flatten(), sequences[j].flatten()))  
    return np.mean(distances)  
  
diversity_score = diversity_metric(generated_sequences)  
print("Diversity Score:", diversity_score)
```



Diversity Score: 0.27514458399717556

NOVELTY METRIC:

✓ Novelty Metric

Novelty is calculated as the average distance between generated and real samples.

```
[ ] def novelty_metric(generated_sequences, real_sequences):
    distances = []
    for gen_seq in generated_sequences:
        gen_seq_flat = gen_seq.flatten()
        real_distances = [euclidean(gen_seq_flat, real_seq.flatten()) for real_seq in real_sequences]
        distances.append(np.mean(real_distances))
    return np.mean(distances)

novelty_score = novelty_metric(generated_sequences, processed_sequences[:n_samples]) # Use n_samples of real data
print("Novelty Score:", novelty_score)
```

Novelty Score: 642.2842038963828

FRECHET AUDIO DISTANCE METRIC:

✓ Fréchet Audio Distance (FAD) Proxy

For the FAD proxy, we calculate the mean squared error between the means and variances of the real and generated distributions as a simplified approximation.

```
[ ] def fad_metric(generated_sequences, real_sequences):
    gen_means = np.mean(generated_sequences, axis=0)
    gen_vars = np.var(generated_sequences, axis=0)

    real_means = np.mean(real_sequences, axis=0)
    real_vars = np.var(real_sequences, axis=0)

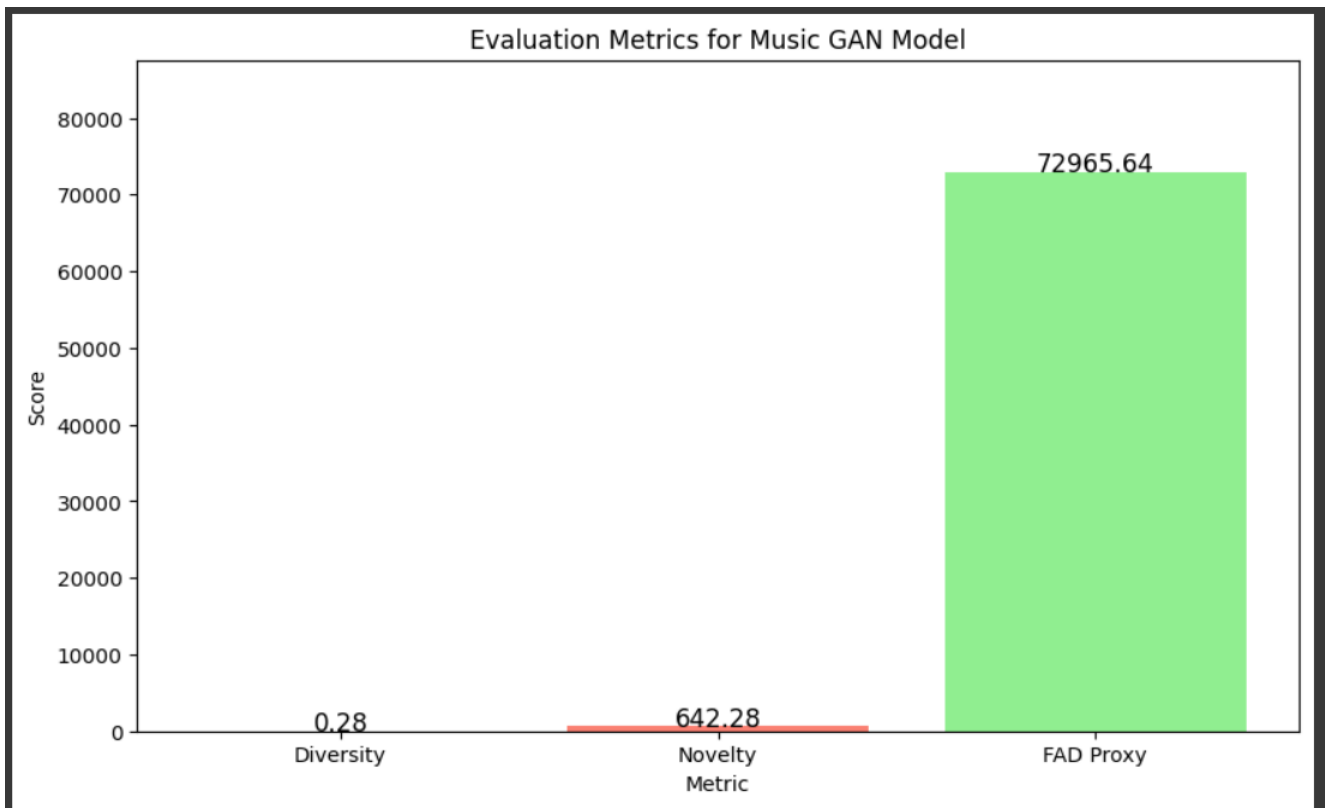
    mean_diff = mean_squared_error(gen_means.flatten(), real_means.flatten())
    var_diff = mean_squared_error(gen_vars.flatten(), real_vars.flatten())

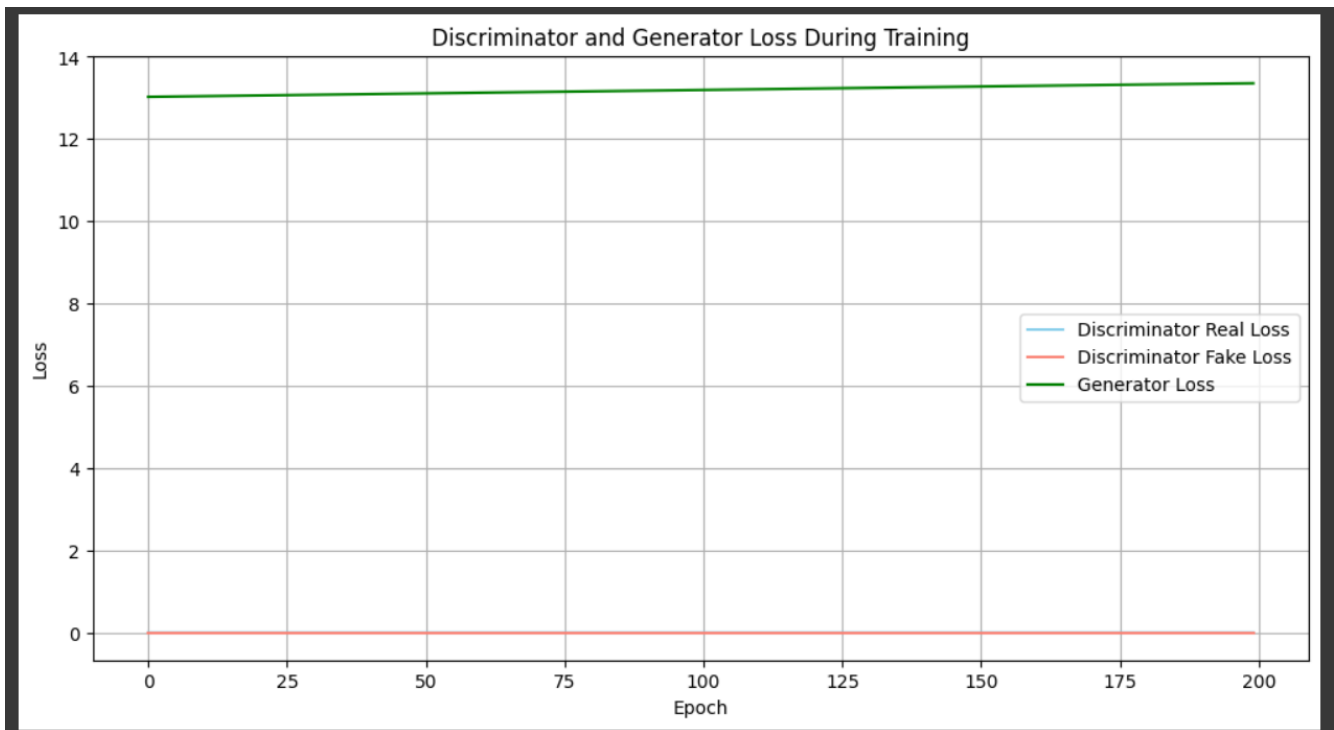
    return mean_diff + var_diff

fad_score = fad_metric(generated_sequences, processed_sequences[:n_samples])
print("FAD Proxy Score:", fad_score)
```

FAD Proxy Score: 72965.63919495112

GRAPH COMPARING THE METRICS:





3.1 GATED RECURRENT UNITS (GRU)

MODEL SUMMARY:

Model: "sequential"

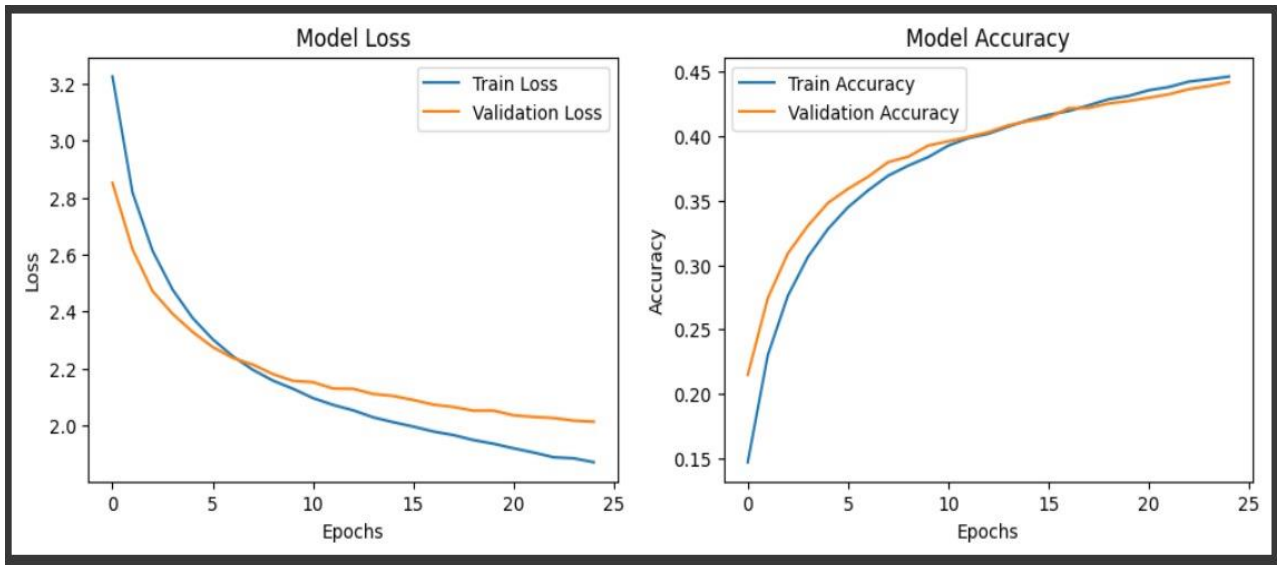
Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 50, 100)	8,700
gru (GRU)	(None, 50, 256)	274,944
dropout (Dropout)	(None, 50, 256)	0
gru_1 (GRU)	(None, 256)	394,752
dense (Dense)	(None, 128)	32,896
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 87)	11,223

Total params: 722,515 (2.76 MB)
Trainable params: 722,515 (2.76 MB)
Non-trainable params: 0 (0.00 B)
None

EPOCHS WITH ACCURACY:

Epoch 10/25	2899/2899	37s	13ms/step	- accuracy: 0.3891	- loss: 2.1029	- val_accuracy: 0.3926	- val_loss: 2.1562
Epoch 11/25	2899/2899	37s	13ms/step	- accuracy: 0.3967	- loss: 2.0703	- val_accuracy: 0.3960	- val_loss: 2.1515
Epoch 12/25	2899/2899	41s	13ms/step	- accuracy: 0.4026	- loss: 2.0501	- val_accuracy: 0.3994	- val_loss: 2.1294
Epoch 13/25	2899/2899	37s	13ms/step	- accuracy: 0.4066	- loss: 2.0256	- val_accuracy: 0.4030	- val_loss: 2.1283
Epoch 14/25	2899/2899	41s	13ms/step	- accuracy: 0.4116	- loss: 2.0100	- val_accuracy: 0.4084	- val_loss: 2.1101
Epoch 15/25	2899/2899	41s	13ms/step	- accuracy: 0.4175	- loss: 1.9857	- val_accuracy: 0.4117	- val_loss: 2.1033
Epoch 16/25	2899/2899	41s	13ms/step	- accuracy: 0.4208	- loss: 1.9710	- val_accuracy: 0.4143	- val_loss: 2.0890
Epoch 17/25	2899/2899	41s	13ms/step	- accuracy: 0.4248	- loss: 1.9508	- val_accuracy: 0.4217	- val_loss: 2.0727
Epoch 18/25	2899/2899	41s	13ms/step	- accuracy: 0.4292	- loss: 1.9382	- val_accuracy: 0.4218	- val_loss: 2.0645
Epoch 19/25	2899/2899	41s	13ms/step	- accuracy: 0.4322	- loss: 1.9256	- val_accuracy: 0.4253	- val_loss: 2.0515
Epoch 20/25	2899/2899	41s	13ms/step	- accuracy: 0.4367	- loss: 1.9106	- val_accuracy: 0.4272	- val_loss: 2.0516
Epoch 21/25	2899/2899	41s	13ms/step	- accuracy: 0.4376	- loss: 1.9012	- val_accuracy: 0.4298	- val_loss: 2.0353
Epoch 22/25	2899/2899	41s	13ms/step	- accuracy: 0.4466	- loss: 1.8755	- val_accuracy: 0.4324	- val_loss: 2.0296
Epoch 23/25	2899/2899	41s	13ms/step	- accuracy: 0.4466	- loss: 1.8624	- val_accuracy: 0.4364	- val_loss: 2.0256
Epoch 24/25	2899/2899	41s	13ms/step	- accuracy: 0.4506	- loss: 1.8535	- val_accuracy: 0.4388	- val_loss: 2.0167
Epoch 25/25	2899/2899	41s	13ms/step	- accuracy: 0.4515	- loss: 1.8464	- val_accuracy: 0.4418	- val_loss: 2.0128

ACCURACY AND LOSS GRAPH:



3.2 VARIATIONAL AUTOENCODERS (VAE)

EPOCHS WITH ACCURACY AND LOSSES:

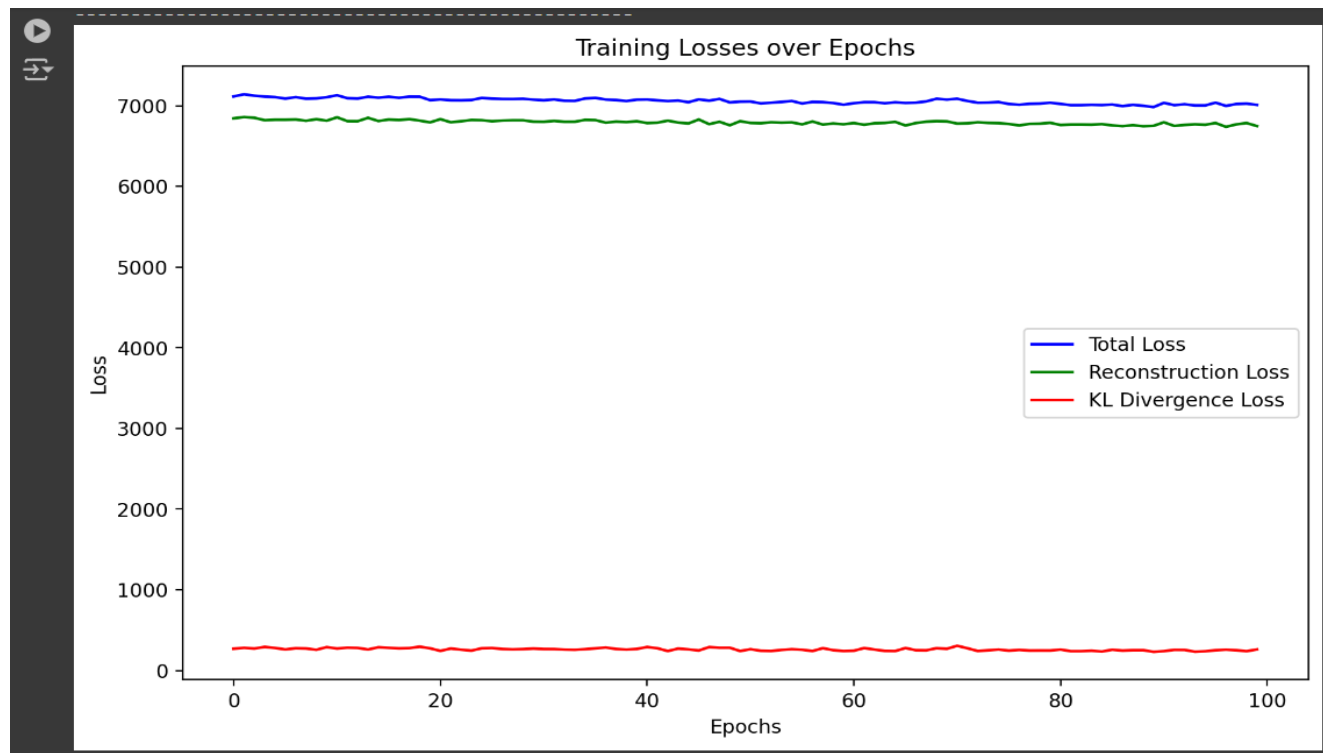
```
➡ Epoch 1/100  
Total Loss: 7113.7299  
Reconstruction Loss: 6842.6339  
KL Divergence Loss: 271.0960  
Accuracy: 0.9220  
-----  
Epoch 2/100  
Total Loss: 7140.2671  
Reconstruction Loss: 6858.3044  
KL Divergence Loss: 281.9627  
Accuracy: 0.9207  
-----  
Epoch 3/100  
Total Loss: 7122.7848  
Reconstruction Loss: 6849.5977  
KL Divergence Loss: 273.1871  
Accuracy: 0.9238  
-----  
Epoch 4/100  
Total Loss: 7112.2157  
Reconstruction Loss: 6819.0656  
KL Divergence Loss: 293.1501  
Accuracy: 0.9247  
-----  
Epoch 5/100  
Total Loss: 7105.5617  
Reconstruction Loss: 6825.3249  
KL Divergence Loss: 280.2368  
Accuracy: 0.9226
```

```
➡ Epoch 6/100  
Total Loss: 7087.0328  
Reconstruction Loss: 6825.1899  
KL Divergence Loss: 261.8429  
Accuracy: 0.9229  
-----  
Epoch 7/100  
Total Loss: 7105.2270  
Reconstruction Loss: 6828.2664  
KL Divergence Loss: 276.9605  
Accuracy: 0.9250  
-----  
Epoch 8/100  
Total Loss: 7086.0243  
Reconstruction Loss: 6812.2180  
KL Divergence Loss: 273.8063  
Accuracy: 0.9226  
-----  
Epoch 9/100  
Total Loss: 7089.6955  
Reconstruction Loss: 6831.9609  
KL Divergence Loss: 257.7346  
Accuracy: 0.9228  
-----  
Epoch 10/100  
Total Loss: 7104.0064  
Reconstruction Loss: 6813.2129  
KL Divergence Loss: 290.7935  
Accuracy: 0.9230
```

```
Epoch 90/100
Total Loss: 6982.2688
Reconstruction Loss: 6749.9246
KL Divergence Loss: 232.3442
Accuracy: 0.9249
-----
Epoch 91/100
Total Loss: 7034.7099
Reconstruction Loss: 6793.7987
KL Divergence Loss: 240.9112
Accuracy: 0.9241
-----
Epoch 92/100
Total Loss: 7006.0280
Reconstruction Loss: 6748.8514
KL Divergence Loss: 257.1765
Accuracy: 0.9263
-----
Epoch 93/100
Total Loss: 7017.2032
Reconstruction Loss: 6760.8133
KL Divergence Loss: 256.3897
Accuracy: 0.9258
-----
Epoch 94/100
Total Loss: 7002.5285
Reconstruction Loss: 6768.7740
KL Divergence Loss: 233.7545
Accuracy: 0.9266
-----
Epoch 95/100
Total Loss: 7002.4142
Reconstruction Loss: 6762.9878
KL Divergence Loss: 239.4264
Accuracy: 0.9265
```

```
Accuracy: 0.9265
-----
Epoch 96/100
Total Loss: 7037.1904
Reconstruction Loss: 6784.8742
KL Divergence Loss: 252.3162
Accuracy: 0.9239
-----
Epoch 97/100
Total Loss: 6996.9036
Reconstruction Loss: 6737.3262
KL Divergence Loss: 259.5774
Accuracy: 0.9251
-----
Epoch 98/100
Total Loss: 7020.0904
Reconstruction Loss: 6767.8088
KL Divergence Loss: 252.2816
Accuracy: 0.9251
-----
Epoch 99/100
Total Loss: 7024.9517
Reconstruction Loss: 6784.6052
KL Divergence Loss: 240.3465
Accuracy: 0.9251
-----
Epoch 100/100
Total Loss: 7009.2905
Reconstruction Loss: 6746.8640
KL Divergence Loss: 262.4265
Accuracy: 0.9243
```

GRAPH FOR TRAINING LOSSES (RECONSTRUCTION LOSS AND KL DIVERGENCE LOSS):



OUTPUT MUSIC GENERATED:

```
# # Convert and print generated music
# for sample in generated_music:
#     notes = [music21.note.Note(int(n * max_midi_value)) for n in sample if n > 0] # Convert back to MIDI number
#     stream = music21.stream.Stream(notes)
#     stream.show('midi') # Display MIDI
```

0:00 -13:23  

0:00 -13:23  

0:00 -13:23  

0:00 -13:23  

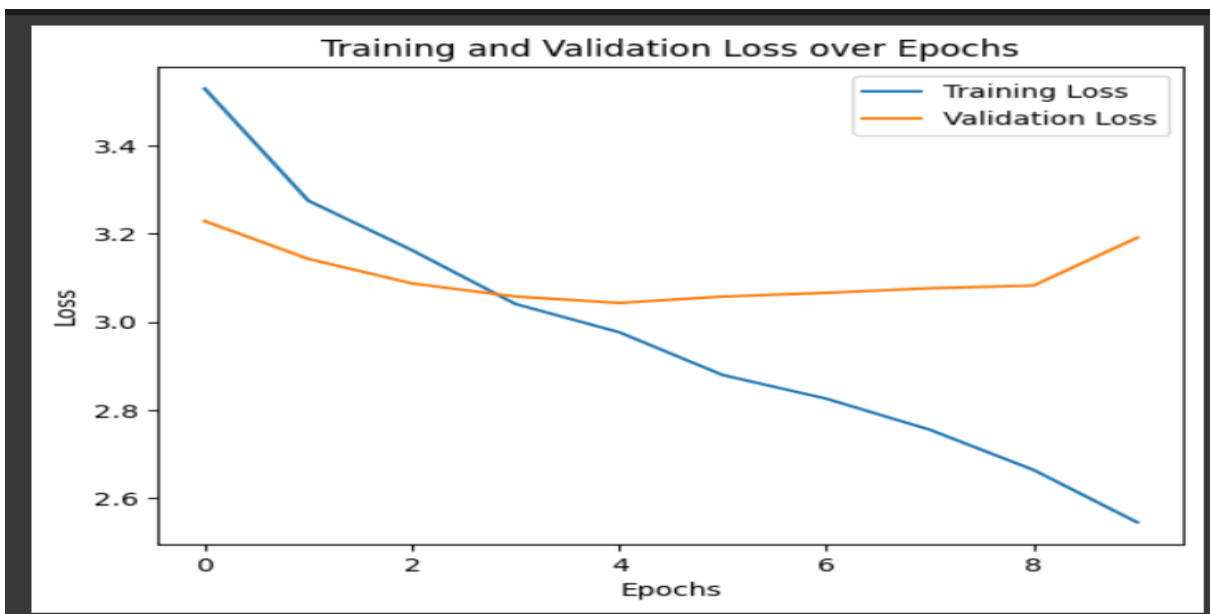
3.3 TRANSFORMERS

EPOCHS WITH LOSS GENERATED:

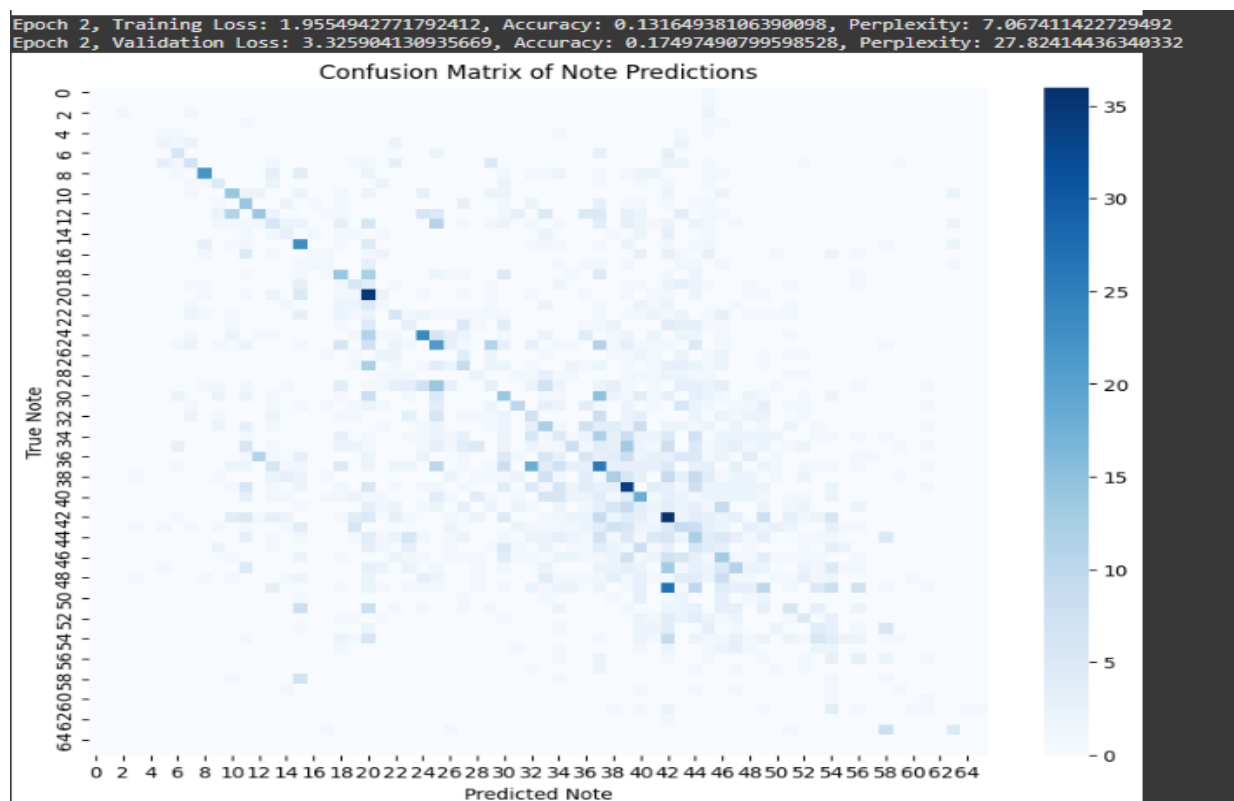
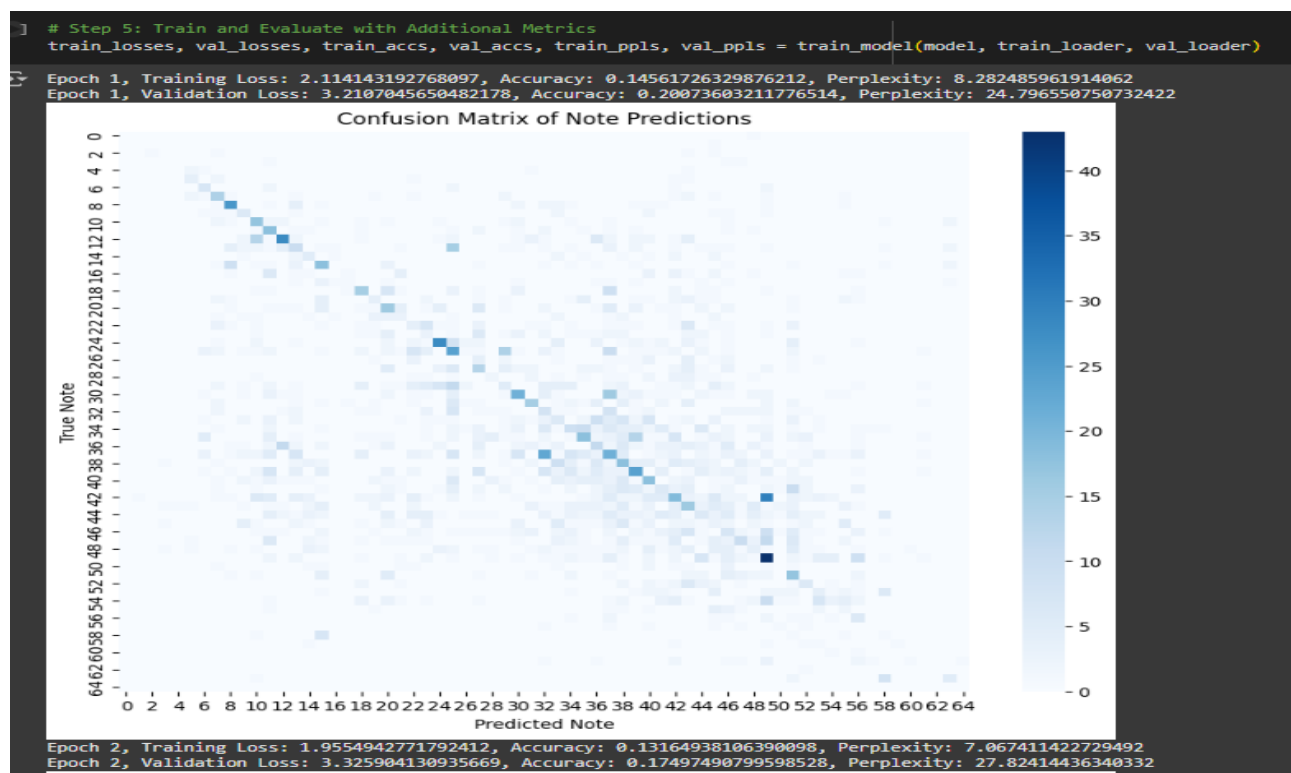
```
[ ] # Step 5: Train and Evaluate
    train_losses, val_losses = train_model(model, train_loader, val_loader)
```

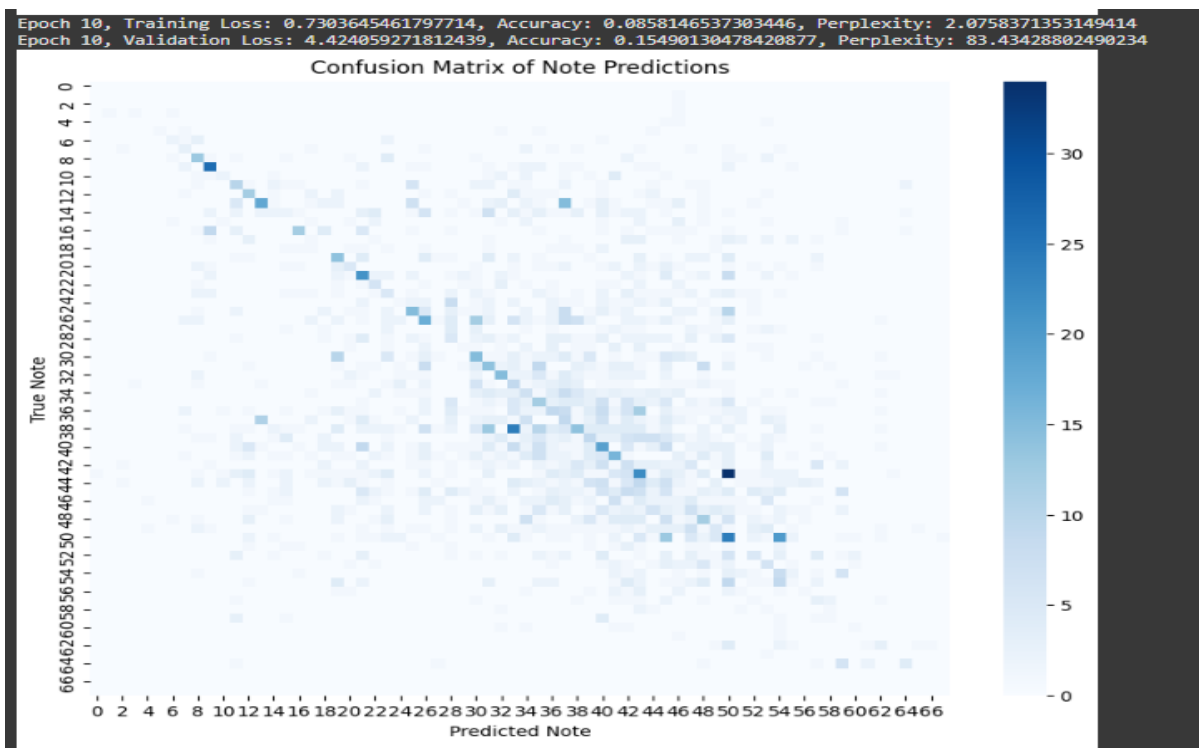
Epoch 1, Training Loss: 3.5286594331264496
Epoch 1, Validation Loss: 3.2282888293266296
Epoch 2, Training Loss: 3.274551957845688
Epoch 2, Validation Loss: 3.142667233943939
Epoch 3, Training Loss: 3.162438452243805
Epoch 3, Validation Loss: 3.0870547890663147
Epoch 4, Training Loss: 3.040376365184784
Epoch 4, Validation Loss: 3.057352602481842
Epoch 5, Training Loss: 2.976456433534622
Epoch 5, Validation Loss: 3.0427441596984863
Epoch 6, Training Loss: 2.8793329894542694
Epoch 6, Validation Loss: 3.057245910167694
Epoch 7, Training Loss: 2.8257194608449936
Epoch 7, Validation Loss: 3.0657646656036377
Epoch 8, Training Loss: 2.7552388310432434
Epoch 8, Validation Loss: 3.0761853456497192
Epoch 9, Training Loss: 2.6642557829618454
Epoch 9, Validation Loss: 3.0821940302848816
Epoch 10, Training Loss: 2.5454383492469788
Epoch 10, Validation Loss: 3.190662443637848

TRAINING AND VALIDATION LOSS EPOCHS:

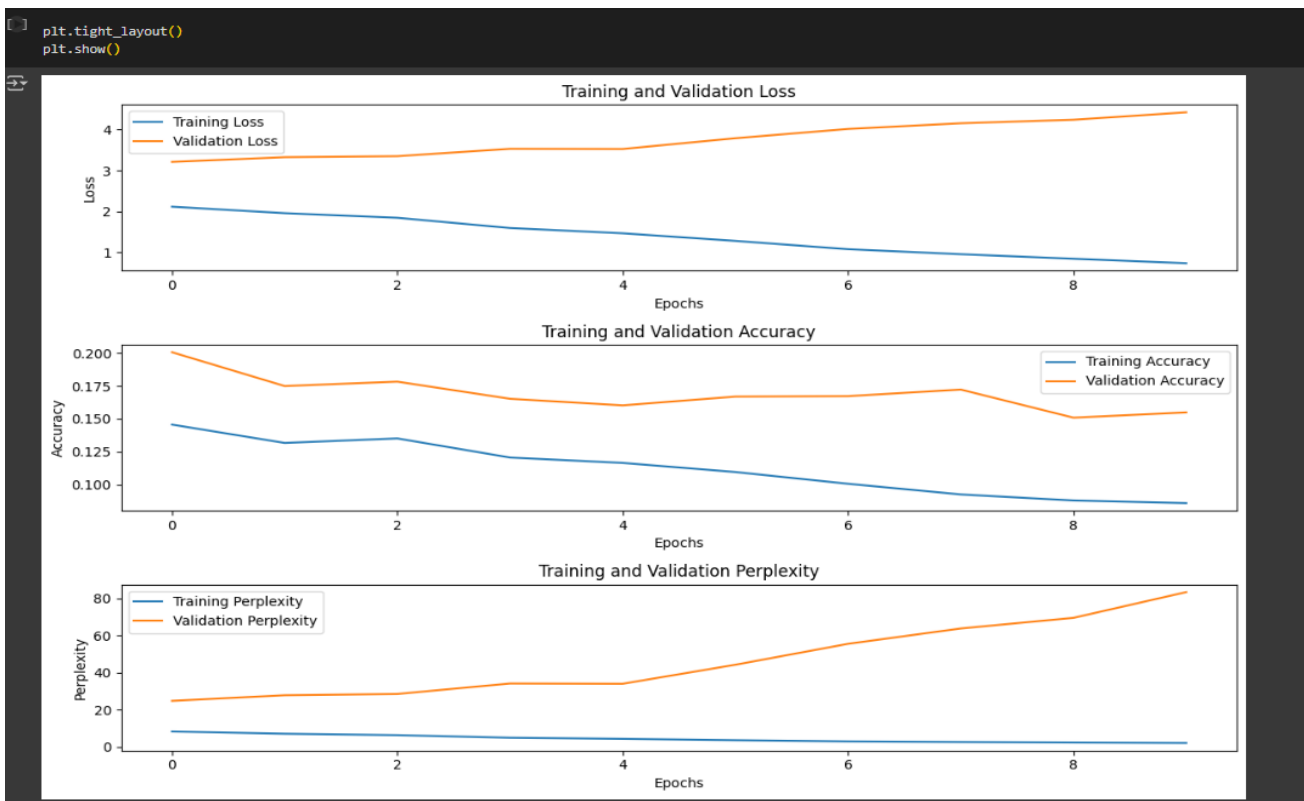


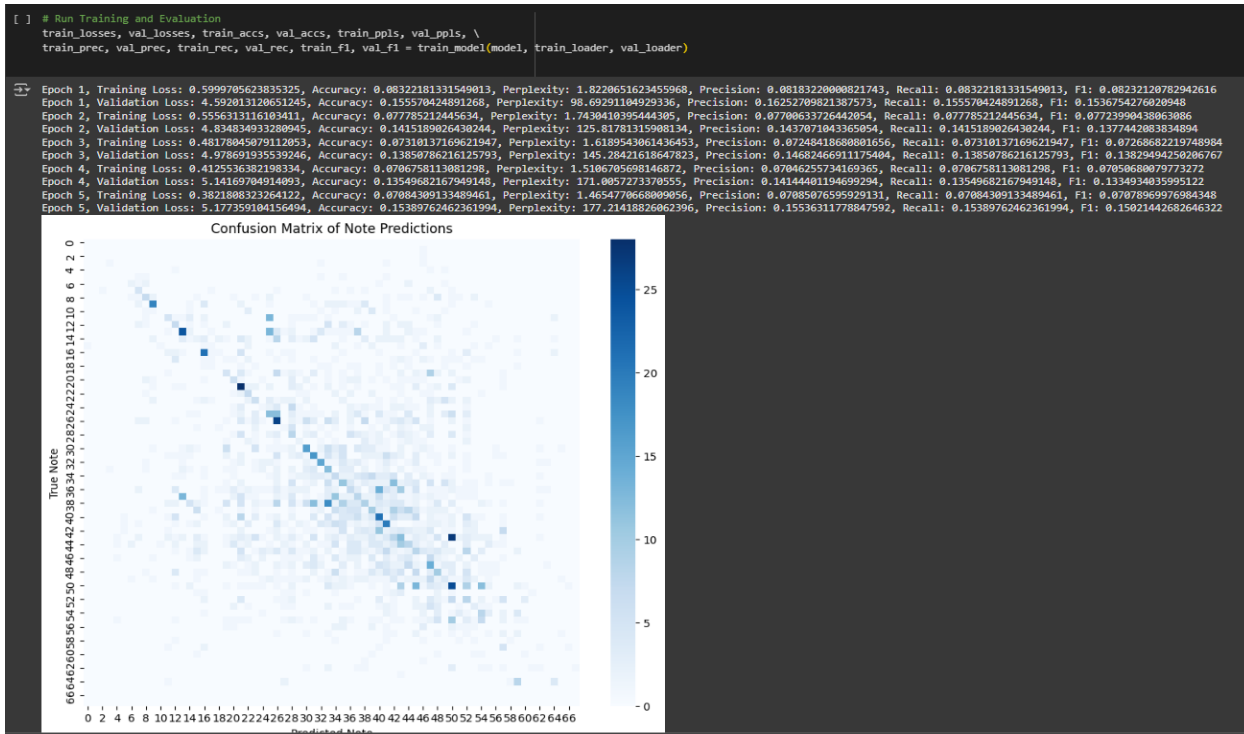
CONFUSION MATRIX FOR EACH EPOCH:



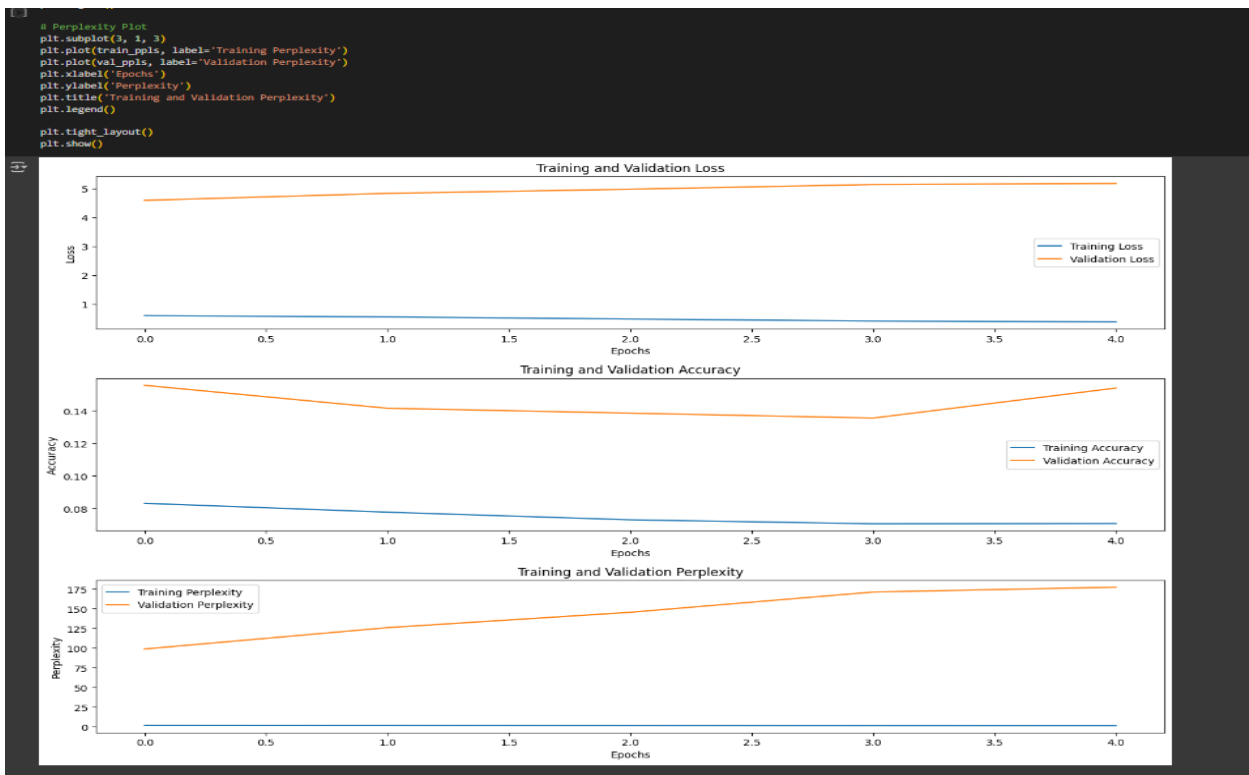


TRAINING AND VALIDATION LOSS, ACCURACY, PERPLEXITY SCORES GRAPH:





PERPLEXITY PLOT:



3.4 OPENAI JUKEBOX

CODE:

```
[ ] your_lyrics = """
We're no strangers to love
You know the rules and so do I
A full commitment's what I'm thinking of
You wouldn't get this from any other guy

I just wanna tell you how I'm feeling
Gotta make you understand

Never gonna give you up
Never gonna let you down
Never gonna run around and desert you
Never gonna make you cry
Never gonna say goodbye
Never gonna tell a lie and hurt you
"""
```

> Select your settings and run this cell to start generating



model: 5b_lyrics

hps.n_samples: 2

hps.name: "/content/gdrive/MyDrive/Project_1"

speed_upsampling: ☒

model: 5b_lyrics

hps.n_samples: 2

hps.name: "/content/gdrive/MyDrive/Project_1"

speed_upsampling: ☒

mode: primed

audio_file: "/content/gdrive/MyDrive/your_file.wav"

prompt_length_in_seconds:

sample_length_in_seconds: 70

select_artist: the beatles

select_genre: pop rock

sampling_temperature: .98

disconnect_runtime_after_finish: ☒

3.5 DIFFUSION MODEL

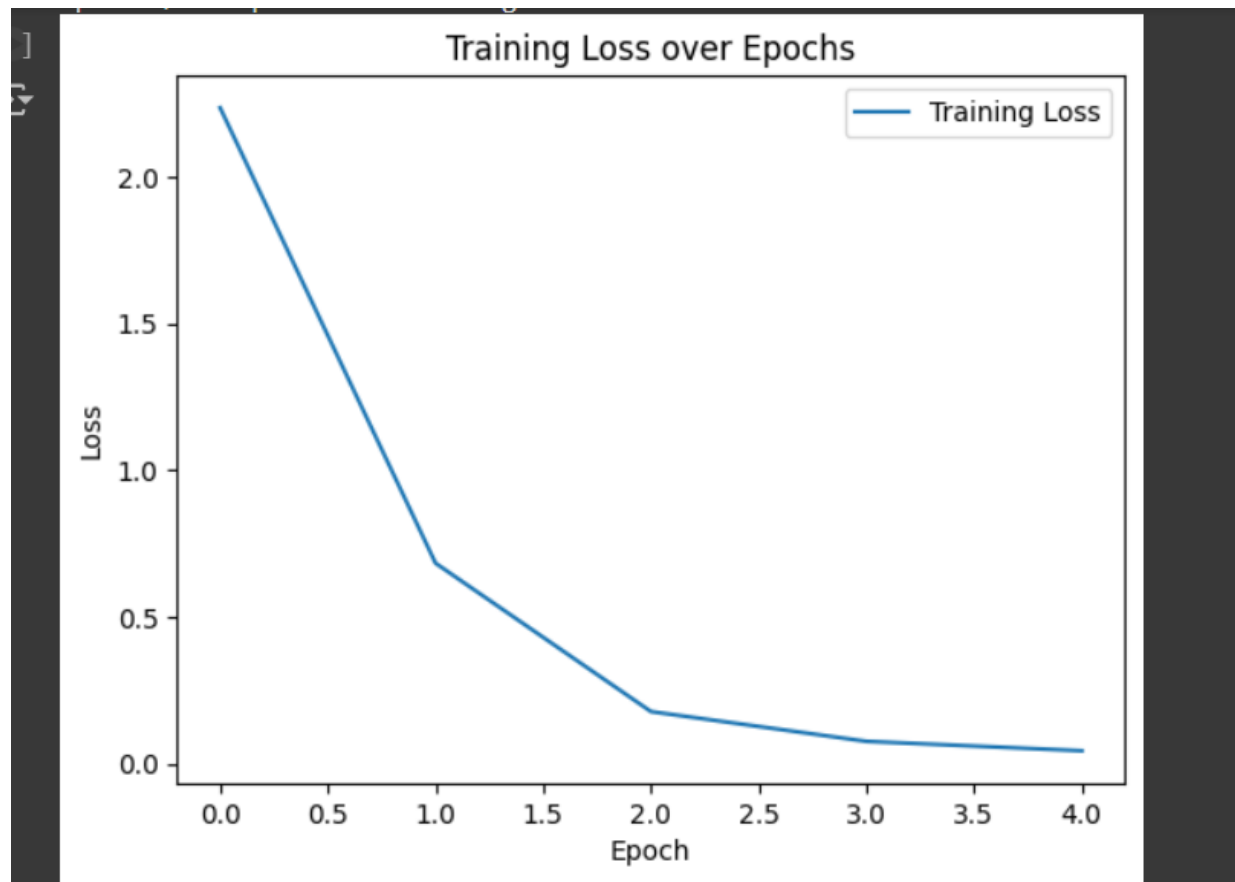
EPOCHS WITH LOSS GENERATED:

```
[ ] # Train the model and plot loss history
losses = train_diffusion_model(diffusion_model, data_loader, optimizer, epochs=5)
plt.plot(losses, label="Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss over Epochs")
plt.legend()
plt.show()
```

```
Epoch 1/5, Batch 1/39, Loss: 5.0506
Epoch 1/5, Batch 2/39, Loss: 4.8374
Epoch 1/5, Batch 3/39, Loss: 4.6079
Epoch 1/5, Batch 4/39, Loss: 4.1066
Epoch 1/5, Batch 5/39, Loss: 4.0643
Epoch 1/5, Batch 6/39, Loss: 3.7566
Epoch 1/5, Batch 7/39, Loss: 3.3774
Epoch 1/5, Batch 8/39, Loss: 3.3039
Epoch 1/5, Batch 9/39, Loss: 3.0599
Epoch 1/5, Batch 10/39, Loss: 2.9869
Epoch 1/5, Batch 11/39, Loss: 2.3285
Epoch 1/5, Batch 12/39, Loss: 2.3614
Epoch 1/5, Batch 13/39, Loss: 2.5306
Epoch 1/5, Batch 14/39, Loss: 1.5516
Epoch 1/5, Batch 15/39, Loss: 2.6006
Epoch 1/5, Batch 16/39, Loss: 1.8207
Epoch 1/5, Batch 17/39, Loss: 1.4267
Epoch 1/5, Batch 18/39, Loss: 1.9473
Epoch 1/5, Batch 19/39, Loss: 1.8736
Epoch 1/5, Batch 20/39, Loss: 1.8262
Epoch 1/5, Batch 21/39, Loss: 2.0337
Epoch 1/5, Batch 22/39, Loss: 1.8672
Epoch 1/5, Batch 23/39, Loss: 2.2173
Epoch 1/5, Batch 24/39, Loss: 1.1752
```

```
Epoch 5/5, Batch 12/39, Loss: 0.0661
Epoch 5/5, Batch 13/39, Loss: 0.0221
Epoch 5/5, Batch 14/39, Loss: 0.0617
Epoch 5/5, Batch 15/39, Loss: 0.0620
Epoch 5/5, Batch 16/39, Loss: 0.0365
Epoch 5/5, Batch 17/39, Loss: 0.1035
Epoch 5/5, Batch 18/39, Loss: 0.0319
Epoch 5/5, Batch 19/39, Loss: 0.0414
Epoch 5/5, Batch 20/39, Loss: 0.0237
Epoch 5/5, Batch 21/39, Loss: 0.0367
Epoch 5/5, Batch 22/39, Loss: 0.0218
Epoch 5/5, Batch 23/39, Loss: 0.0373
Epoch 5/5, Batch 24/39, Loss: 0.0664
Epoch 5/5, Batch 25/39, Loss: 0.0360
Epoch 5/5, Batch 26/39, Loss: 0.0288
Epoch 5/5, Batch 27/39, Loss: 0.0317
Epoch 5/5, Batch 28/39, Loss: 0.0518
Epoch 5/5, Batch 29/39, Loss: 0.0360
Epoch 5/5, Batch 30/39, Loss: 0.0309
Epoch 5/5, Batch 31/39, Loss: 0.0577
Epoch 5/5, Batch 32/39, Loss: 0.0233
Epoch 5/5, Batch 33/39, Loss: 0.0292
Epoch 5/5, Batch 34/39, Loss: 0.0372
Epoch 5/5, Batch 35/39, Loss: 0.0398
Epoch 5/5, Batch 36/39, Loss: 0.0396
Epoch 5/5, Batch 37/39, Loss: 0.0680
Epoch 5/5, Batch 38/39, Loss: 0.0207
```

TRAINING LOSS OVER EPOCHS:



RECONSTRUCTION ACCURACY:

```
[ ] reconstruction_accuracy(diffusion_model, data_loader)
```

```
⇒ Reconstruction Accuracy: 99.94%  
0.9993979061880499
```

PERPLEXITY SCORE:

```
[ ] calculate_perplexity(diffusion_model, data_loader)
```

```
⇒ Perplexity: 1.03  
1.0338762688454641
```

MODEL SUMMARY:



Layer (type)	Output Shape	Param #
Embedding-1	[-1, 1, 128, 256]	32,768
Linear-2	[-1, 1, 128, 256]	65,792
Linear-3	[-1, 1, 128, 256]	65,792
Linear-4	[-1, 1, 128, 128]	32,896
DiffusionModel-5	[-1, 1, 128, 128]	0

Total params: 197,248
Trainable params: 197,248
Non-trainable params: 0

Input size (MB): 0.00
Forward/backward pass size (MB): 1.00
Params size (MB): 0.75
Estimated Total Size (MB): 1.75

Chapter 3

Conclusion and Future Work

This project hence brings into limelight potential transformation from artificial intelligence in the creative arts with an exploration of deep learning architectures for music composition. It guides how six state-of-the-art models with capability and limitations, namely GANs, GRUs, VAEs, Transformers, Diffusion Models, and OpenAI Jukebox would be helpful in people understanding the opportunity in its true potential.

Every model demonstrated one significant strength: GANs showed a strong ability to generate realistic musical textures, the GRUs demonstrate high qualities when modeling temporal dependencies, and VAEs allow creative diversity. In addition, transformers demonstrate their strengths in dealing with long-range dependencies, in generating intricate compositions, and Diffusion Models demonstrate high qualities also in this regard. Jukebox is superior in the generation of high-fidelity audio.

Therefore, this paper highlights growing importance of subjective metrics such as human appreciation and emotional resonance in that manner to traditional objective measures of harmonicity and rhythmic coherence. Other than verifying the usefulness of AI in music composing, these results go further and show that AI has great potential in complementing human creativity. Apart from these challenges in the aspect of computational resources demands, standardizing the evaluation, or even how to balance novelty with coherence, the questions of efficiency and quality continue to persist. Ultimately, this project drives home the point that AI models are partners, not only tools, in this creative process-be it a future of innovative and diverse possibility in shaping music.

The experiments conducted within this endeavor indicate how much more research and tuning is needed toward making AI-based music composition a reality. Research can be extended to hybrid models, bringing together strengths from several architectures-typically coherence in the case of GRUs, or creativity variability in the case of VAEs or fidelity in the case of Jukebox. This could make it possible to design systems that deliver compositions which are at once both structurally sound and artistically inspiring.

Another important avenue relates to the further development of the kinds of metrics to evaluate this music. For instance, by incorporating real-time feedback from musicians and listeners, one may better understand emotional and cultural reactions to AI-generated music. More models might be created for genres or regional musical styles that have been unrepresented to date, enhancing the inclusiveness and diversity of AI-generated music.

From AI music systems, the applications include its incorporation into interactive systems: some examples are real-time improvisation tools, adaptive game soundtracks, or collaborative platforms for composers. It has primarily arisen in terms of challenges: one is to increase computational efficiency, and the second is to make the models highly interpretable, both of which will be critical to making this technology more practical and widely adopted. Rapid evolution of AI will surely be further enhancing musical composition, provided only as an enabler of new forms of creative expression rather than a substitute for human artistry

Appendices

<

Appendix A: Models Summary

Generative Adversarial Networks (GANs):

adversarial training method comprising a generator and a discriminator that make fully convincing pieces of music.

Gated Recurrent Units (GRUs):

Modelling sequences that capture compositional inductive biases that reflects temporal dependencies.

Variational Autoencoders (VAEs):

Probabilistic models that enable creative opportunities for explorations through musical variations which are diverse.

Transformers:

Attention-based architectures that excel at long-range dependencies as well as complex structures.

Diffusion Models:

New method that refines noise into coherent outputs for high-resolution music generation.

OpenAI's Jukebox is a transformer-based architecture to generate raw audio of high fidelity, including vocal and instrumental music.

Appendix B: Datasets Used

Symbolic Music Dataset:

This dataset has been used for training GANs, GRUs, and VAEs. On the dataset, importance is given to melodies as well as polyphonic music.

Raw Audio Dataset. This dataset is used by OpenAI's Jukebox and Diffusion Models for audio synthesis at good quality.

Custom MIDI Dataset:

The dataset encompasses various genres in order to test how the models may learn and generalize towards different musical styles during training and testing.

Appendix C: Metrics Used for Evaluation

Harmonicity. This talks about the harmonic relationship of every composition.

Rhythmic Consistency. It measures uniformity and therefore, temporal alignment.

Novelty measures the difference of compositions from the training corpus.

Subjective Human Evaluation: Listening-based feedback regarding musicality, coherence and emotional strength.

Appendix D: Tools and Libraries

Music21: the toolkit for music analysis; we use it for symbolic data processing.

Pretty MIDI: makes handling the MIDI files and extracting features easier.

TensorFlow and PyTorch are the frame works to develop and train deep learning models
Matplotlib and Seaborn are used for visualizing the model outputs and its evaluation.

REFERENCES

- [1] Chou, S. Y., et al. (2017). MuseGAN: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment. *arXiv*. <https://arxiv.org/abs/1709.06298>
- [2] Donahue, C., et al. (2018). WaveGAN and SpecGAN: Audio synthesis with generative adversarial networks. *arXiv*. <https://arxiv.org/abs/1811.09913>
- [3] OpenAI. (2020). Jukebox: A generative model for music. *arXiv*. <https://arxiv.org/abs/2005.00341>
- [4] Huang, C. Z. A., et al. (2018). Music transformer: Generating music with long-term structure. *arXiv*. <https://arxiv.org/abs/1809.04281>
- [5] Kumar, K., et al. (2019). MelGAN: Generative adversarial networks for conditional waveform synthesis. *arXiv*. <https://arxiv.org/abs/1910.06711>
- [6] Van den Oord, A., et al. (2017). Neural discrete representation learning. *arXiv*. <https://arxiv.org/abs/1711.00937>
- [7] Eck, D., & Schmidhuber, J. (2002). Learning the long-term structure of the blues. In *Proceedings of the International Conference on Artificial Neural Networks*.
- Lewandowski, S., et al. (2012). Polyphonic music composition with recurrent neural networks. *ResearchGate*. https://www.researchgate.net/publication/375671182_Music_Generation_using_RNN-LSTM_with_GRU
- [8] Kingma, D. P., & Welling, M. (2013). Auto-encoding variational Bayes. *arXiv*. <https://arxiv.org/abs/1312.6114>

